

Analysis and Redesign of Variable-Length Data Storage in HDF5

adam.ohlson@lifeboat.llc

john.mainzer@lifeboat.llc

This document provides an initial investigation into the implementation and lifecycle of variable-length (VL) datatypes in HDF5 under the native VOL connector. It examines how VL data is represented, stored, and managed across memory and file systems, including datatype definition, I/O paths, heap-based storage, and memory reclamation. Attention is given to the reliance on global heap storage and its implications for performance, caching behavior, and SWMR consistency. This analysis serves as the foundation for future work aimed at integrating VL payload storage into structured chunk-based layouts. This document also evaluates potential approaches for improving VL storage, particularly in structured chunk datasets.

- 1 Introduction 5
- 2 Requirements..... 5
 - 2.1 Functional Requirements..... 5
 - 2.2 Design Considerations 6
- 3 Current Implementation..... 6
 - 3.1 Variable-Length Datatype Semantics 6
 - 3.2 Storage Model: Global Heap..... 7
 - 3.3 I/O Behavior..... 9
 - 3.4 Memory Management..... 9
 - 3.5 Variable-Length Data Lifecycle 10
 - 3.5.1 Actors and Artifacts 10
 - 3.5.2 Create / Define the VL Datatype..... 11
 - 3.5.3 Write Path: Memory -> File..... 14
 - 3.5.4 On-Disk Storage and Caching..... 18
 - 3.5.5 Read Path: File -> Memory 19
 - 3.5.6 Memory Management and Reclaim..... 22
 - 3.5.7 Delete / Overwrite / Object Lifetime..... 23
 - 3.5.8 SWMR Visibility Limitations..... 23
- 4 Global Heap (H5HG) Analysis..... 24
 - 4.1 Heap Structure and Layout 25
 - 4.2 Allocation and Free Space Management..... 27
 - 4.3 Removal and Compaction..... 28
 - 4.4 Extension and Growth 29
 - 4.5 Metadata Cache Interaction..... 30
 - 4.6 Implications for Redesign 31
- 5 Proposed Designs..... 32
 - 5.1 Flattened Chunk-Local Model..... 32
 - 5.1.1 Deeper Implementation Analysis 34
 - 5.1.2 Implementation Requirements and Responsibilities 35
 - 5.2 Chunk-Local Heap Model..... 36
 - 5.2.1 Mapping of the Global Heap Components to Chunk-Local Design..... 39
 - 5.2.2 Integration with Structured Chunk Pipeline 40

5.2.3	Versioning and Upgrade Path.....	42
5.2.4	Fractal Heap Storage Considerations	42
5.3	Discussion and Design Tradeoffs	43
6	Testing.....	44
6.1	Existing Test Coverage	44
6.1.1	Global Heap (H5HG) Tests	44
6.1.2	VL Datatype Tests	45
6.1.3	Structured Chunk Tests.....	45
6.2	New Test Requirements	45
6.3	Summary.....	46
7	Recommendation	46
8	Chunk-Local VL Heap Implementation Mapping.....	47
8.1	Implementation Overview.....	48
8.2	Conceptual Overview and Responsibility Split	49
8.3	Data Structures	50
8.3.1	Decoded Chunk State	50
8.3.2	Operation-Local VL Chunk Context.....	52
8.3.3	Ownership Rules	54
8.3.4	Decoded Local Heap RAM Format.....	54
8.4	Structured-Chunk Section Layout and Descriptor Format	55
8.4.1	H5_SECTION_VL Image Format.....	56
8.4.2	Structured-Chunk VL Descriptor Format	56
8.4.3	Non-Empty VL Heap Layout.....	57
8.4.4	Zero-Sized VL Section.....	57
8.4.5	Empty Chunk Rule.....	58
8.4.6	File-Format Header Alignment	58
8.5	Theory of Operation	59
8.5.1	New Chunk.....	60
8.5.2	Decode	60
8.5.3	Scatter Memory Read.....	61
8.5.4	Gather Memory Write.....	62
8.5.5	Overwrite.....	63
8.5.6	Erase.....	63

8.5.7	Fill.....	64
8.5.8	Encode and Flush	64
8.5.9	Delete Chunk.....	65
8.5.10	Evict and Cleanup	65
8.5.11	Condense and Size Finalization.....	65
8.5.12	Encode-in-Place Policy.....	66
8.5.13	Vector I/O Policy.....	66
8.5.14	SWMR Implications.....	66
8.6	Detailed Code Mapping	66
8.6.1	Summary of Code Areas	66
8.6.2	H5T Chunk-Local VL Backend.....	68
8.6.3	H5T Callback Pseudocode	70
8.6.4	Backend Selection.....	74
8.6.5	Operation-Local Context Passing.....	75
8.6.6	H5HG Local Heap Reuse	76
8.6.7	H5HG Local Helper Algorithms	77
8.6.8	SCC and H5D Structured-Chunk Changes	81
8.6.9	File-Format Persistence Points	85
8.6.10	Cache Admission and Request Sizing	85
8.7	Integrity, Filtering, and Section Metadata Policy	86
8.8	Initial Implementation Policy Decisions	87
8.8.1	Initial Decision Table.....	87
8.8.2	Empty Chunk and Empty Heap Policy.....	88
8.8.3	Versioning Policy.....	89
8.8.4	Compatibility Policy	89
8.9	Implementation Order	90
9	Acknowledgement	91
10	References	91
11	Revision History	91

1 Introduction

A variable-length datatype in HDF5 is a datatype whose elements are sequences of another datatype (the base type), where each element's sequence length may differ. The key implementation detail is that the payload is not stored in line with the dataset's raw element storage. Instead, the dataset stores fixed-size descriptors for each element, and the actual variable-sized data is stored indirectly (typically in heap storage). This allows dataset elements to remain a fixed size while supporting "ragged" data.

2 Requirements

This section defines the functional requirements and key design considerations for improving variable-length datatype handling in HDF5. These requirements are derived from the behavior and limitations of the current implementation, particularly the use of global heap storage for VL payloads.

The goal is to establish a clear set of constraints and expectations that any revised VL storage approach must satisfy, ensuring correctness, compatibility, and performance across a range of use cases, including nested VL datatypes and chunked dataset layouts.

2.1 Functional Requirements

Any revised approach to variable-length datatype storage must preserve the existing semantics of VL types in HDF5, including support for both VL sequences and VL strings. The design must correctly handle nested VL datatypes, such as VL values embedded within compound datatypes or nested VL structures, without introducing ambiguity or loss of information.

The solution must remain fully compatible with the existing dataset I/O interfaces, including H5Dread and H5Dwrite, and must integrate with the current datatype conversion framework. In addition, it must maintain correct memory management behavior, including allocation and reclamation of VL payloads through existing mechanisms such as H5Dvlen_reclaim.

The design should operate within the context of chunked dataset storage, where variable-length data can be managed in conjunction with chunk-local representations. Existing global-heap-based mechanisms will remain in place, particularly for contiguous datasets, and any new approach must coexist with these mechanisms without breaking compatibility with existing files.

Dataset creation for the new structured-chunk VL path should follow the public API direction in the sparse-data API RFC. Structured-chunk storage is selected through H5Pset_struct_chunk(), which records chunk dimensions and a structured-chunk storage flag. H5D_SPARSE_CHUNK selects sparse structured-chunk behavior, H5D_VL_CHUNK selects the chunk-local VL storage path, and the current clarification is that sparse VL uses H5D_SPARSE_CHUNK / H5D_VL_CHUNK. H5D_VL_CHUNK should be understood as a storage-selection flag, not as the datatype marker itself; the datatype graph already determines whether a dataset contains VL data.

The revised storage approach must preserve correct behavior under Single Writer Multiple Reader (SWMR) constraints, ensuring that changes do not violate existing expectations around dataset consistency, visibility, or ordering of operations.

2.2 Design Considerations

The current implementation of VL datatypes relies on global heap storage for payload data, while dataset storage contains only fixed-size descriptors. This separation introduces several important design considerations. Because payloads are stored out-of-line, there is reduced locality between dataset elements and their associated data, which can negatively impact access patterns and I/O efficiency.

Accessing VL data requires additional indirection through heap structures, increasing the number of operations required to read or write a single dataset element. In workloads with many VL elements, this can lead to significant metadata traffic, as global heap objects are managed through the metadata cache rather than through raw data pathways.

Nested VL datatypes introduce additional complexity, as reading or writing a single logical element may require recursive traversal and multiple levels of allocation and parsing. Under SWMR constraints, maintaining consistency between dataset descriptors and heap-resident payloads becomes more complex, since updates must ensure that readers observe a coherent view of both descriptor and payload data.

These considerations motivate the exploration of alternative storage strategies that improve locality, reduce indirection, and simplify consistency management while preserving existing datatype semantics.

3 Current Implementation

Variable-length datatypes introduce additional complexity compared to fixed-size types, as they rely on indirection, dynamic memory management, and coordination between datatype logic and storage backends. The current implementation relies on a descriptor-based model, where dataset elements store fixed-size descriptors that reference separately allocated payload data.

In the context of dataset I/O, variable-length data is processed through the datatype conversion and I/O pipelines, where user-provided buffers are transformed into file representations and vice versa. Variable-length data must ultimately be translated between in-memory representations and on-disk format through serialization and deserialization steps.

The following subsections describe the key aspects of this implementation, including datatype semantics, storage layout, I/O behavior, and memory management.

3.1 Variable-Length Datatype Semantics

Internally, every datatype in HDF5 is represented by an `H5T_t` object, which serves as a wrapper around a shared datatype representation (`H5T_shared_t`). The `H5T_shared_t` structure contains the core definition of the datatype, including its class, size, and class-specific metadata. For variable-length datatypes, this metadata is stored in `shared->u.vlen`, which defines the VL subtype and associated behavior.

VL datatypes rely on a class-specific callback table that defines how VL values are handled in different contexts, including memory and disk representations. These callbacks govern operations such as reading, writing, and reclaiming VL data, and allow the implementation to adapt behavior depending on the storage backend and execution context.

Importantly, the VL datatype itself does not store payload data. Instead, it defines how dataset elements should be interpreted, with the actual payload handled separately through the storage mechanisms described in the following sections.

Although VL data is conceptually variable in size, each dataset element always occupies a fixed number of bytes in dataset storage, as defined by `shared->size`. For VL datatypes, this fixed size corresponds to a descriptor rather than the payload itself. The payload is stored separately, and the descriptor provides the necessary information to locate and interpret that payload.

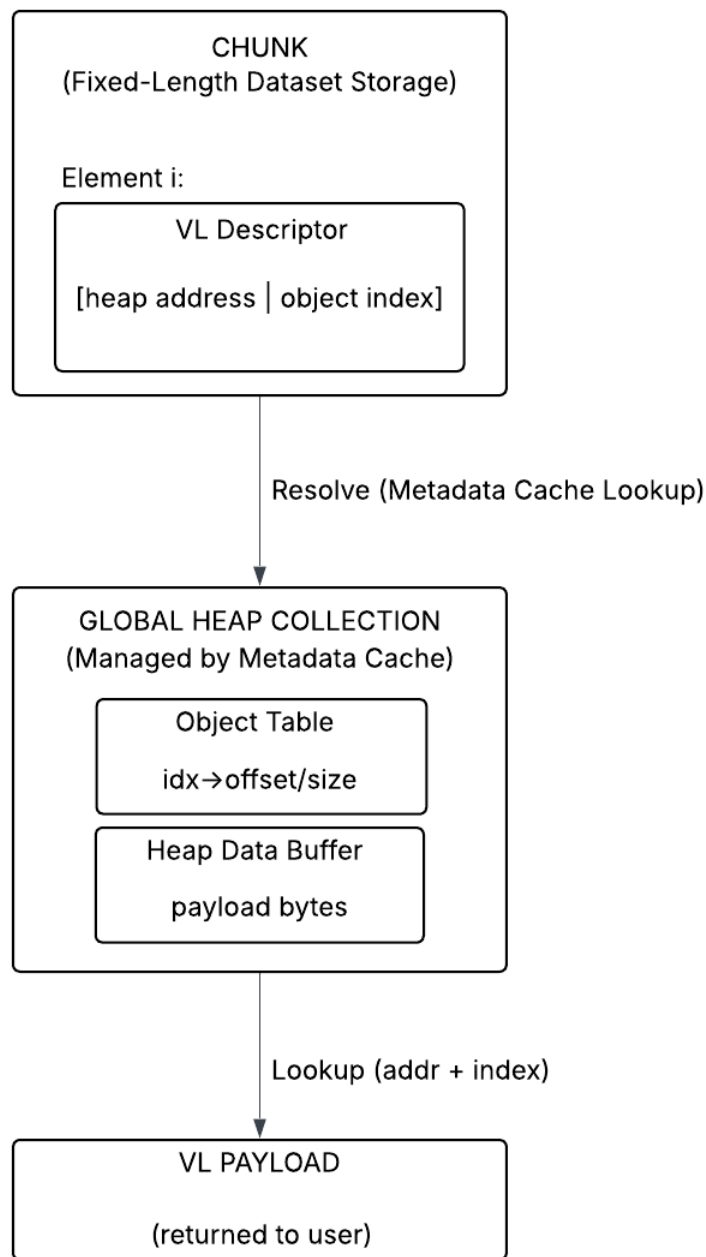
3.2 Storage Model: Global Heap

Under the current HDF5 implementation, VL datatypes store their payload data in global heap objects, while datasets themselves store only fixed-size VL descriptors as part of their raw storage. Each dataset element occupies a constant number of bytes determined by the datatype's `shared->size`, and for VL datatypes this size corresponds to a descriptor containing the sequence length and a reference to a heap object holding the actual payload.

To access the payload, the library must resolve the descriptor through the global heap subsystem. This involves locating the appropriate heap collection via the metadata cache, using the encoded reference to identify the correct heap object, and then retrieving the payload bytes from the heap buffer. The result of this process is the VL payload returned to the user.

This two-stage access pattern highlights a key characteristic of the current implementation: while descriptors are co-located with dataset storage, the payload data itself resides in a separate metadata-managed structure. As a result, accessing VL data requires both dataset I/O and additional heap resolution, introducing indirection and reducing locality between descriptors and their associated payloads.

The current storage model for VL data outlined here is summarized by the following diagram. This diagram describes the legacy descriptor-to-heap VL path conceptually. Though later sections focus on structured chunks, the same descriptor to out-of-line payload pattern also applies to non-structured layouts such as contiguous datasets.



For chunked datasets, VL descriptors are stored within the dataset's chunk data, but the corresponding payloads are stored independently in one or more global heaps. These global heap objects are managed through the metadata cache and are not scoped to individual dataset chunks. As a result, VL payload storage is decoupled from chunk boundaries and chunk-level I/O operations.

When VL data is written, the library allocates a new heap object large enough to hold the payload, writes the payload data into that heap object, and stores a reference to it in the dataset element's descriptor. If an existing VL value is overwritten, a new heap object is allocated and the descriptor is updated to reference it; the previous heap object may then be released. VL data is never resized in place.

When the VL data is read, the library reads the fixed-size descriptor from the dataset, determines the sequence length, dynamically allocates memory for the payload using either the system allocator or a user-defined allocator supplied via the dataset transfer property list, and then reads the payload data from the heap into the allocated buffer. The user's buffer therefore contains fixed-size VL descriptors such as `hvl_t` structures that reference dynamically allocated memory.

Because VL payloads are dynamically allocated and may be nested within compound or other VL datatypes, memory associated with VL data must be reclaimed using `H5Dvlen_reclaim`, which ensures that all allocations are properly released according to the datatype structure and configured memory management routines.

3.3 I/O Behavior

During dataset I/O, variable-length datatypes are handled through the datatype conversion framework, which operates between the memory datatype provided by the user and the file datatype stored in the dataset. When `H5Dwrite` is invoked, the library constructs a conversion path (`H5T_path_t`) from the memory datatype to the file datatype. If the file datatype is variable-length or contains VL fields, the conversion process invokes VL-specific write callbacks through the datatype's `vlen` class. These callbacks allocate storage for the payload using the underlying storage backend, write the payload data, and encode a descriptor into the dataset's raw storage that references the newly created payload.

When `H5Dread` is invoked, the library constructs a conversion path from the file datatype to the memory datatype. For each selected element, the VL descriptor is read from dataset storage and interpreted using VL-specific callbacks. The library determines the payload size, allocates memory using either the default allocator or a user-defined memory manager, and retrieves the payload data from the underlying storage. If necessary, base datatype conversion is performed on the payload elements as part of the conversion process.

Cleanup of VL data in memory is performed using `H5Dvlen_reclaim`, which traverses the datatype graph associated with the memory datatype, identifies all VL descriptors, and invokes the appropriate deallocation routines for each payload.

VL datatype conversion is not a simple in-place transformation. Because VL values may require dynamic allocation, deep traversal of nested structures, and interaction with backend storage, conversion often involves allocating new buffers, recursively converting base elements, and reconstructing descriptors. As a result, VL conversions are typically not no-op operations, even when source and destination datatypes appear similar.

3.4 Memory Management

Variable-length data requires explicit memory management due to the use of dynamically allocated payload buffers during dataset reads. When `H5Dread` encounters a VL datatype, it reads the fixed-size descriptor from the dataset, determines the sequence length, allocates memory for the payload using either the system allocator or a user-defined allocator supplied via the dataset transfer property list, and then reads the payload data into that newly allocated buffer. The user's buffer therefore contains fixed-size VL descriptors (such as `hvl_t` structures) that point to dynamically allocated memory. Because

this allocation is performed by the library and may involve nested VL datatypes or VL fields inside compound datatypes, the memory cannot be safely released with a simple `free()`. Instead, `H5Dvlen_reclaim` must traverse the datatype graph rooted at the memory datatype, locate all VL descriptors within the buffer, and invoke the correct deallocation routines for each payload before zeroing descriptors. This traversal-based reclaim process ensures that all VL allocations, including deeply nested structures and those allocated through custom memory managers, are released correctly and without leaks.

3.5 Variable-Length Data Lifecycle

This subsection traces the lifecycle of variable-length data through the current implementation, from datatype definition through write, storage, read, reclaim, overwrite, and visibility behavior. The goal is to show how the fixed-size descriptor and out-of-line payload interact across the major phases of VL handling.

3.5.1 Actors and Artifacts

Under the current native VOL connector, a VL dataset element stored on disk consists of a fixed-size descriptor embedded directly in the dataset's raw storage, while the actual payload bytes are stored separately in a global heap object. When a VL datatype is set to disk location (`H5T_LOC_DISK`), the element size is defined explicitly as the sum of four bytes for the encoded sequence length plus the size of a VOL-defined blob identifier:

```
/* H5T__vlen_set_loc (H5T_LOC_DISK case) */
dt->shared->size = 4 + cont_info.blob_id_size;
```

This guarantees that each dataset element occupies a constant number of bytes regardless of the size of the associated payload.

The first four bytes of the descriptor encode the sequence length as a 32-bit unsigned integer using `UINT32_ENCODE` and `UINT32_DECODE`. The remaining bytes form the blob identifier, whose size is provided by the VOL connector through the `H5VL_FILE_GET_CONT_INFO` operation. The datatype layer treats this blob identifier as opaque and does not interpret its internal structure. Instead, payload operations are routed through the VOL “blob” interface (`H5VL_blob_put`, `H5VL_blob_get`, and `H5VL_blob_specific`), allowing the active connector to determine how and where the payload is stored.

The VOL discussion is included only to explain the current storage handoff. VL conversion reaches payload storage through the VOL blob interface before the native connector maps that blob operation to H5HG global heap storage. Other VOL connectors may choose different payload storage internally. The structured-chunk VL design changes the native endpoint. It does not require a new external VL API or require every VOL connector to use the same heap representation.

In the native VOL connector, the blob identifier encodes a global heap object ID. When writing a VL value to disk, the native blob implementation allocates storage in the global heap via `H5HG_insert`:

```

/* H5VL__native_blob_put */
if (H5HG_insert(f, size, buf, &hobjid) < 0)
    HGOTO_ERROR(...);

```

The resulting heap address and index are then encoded into the blob identifier portion of the descriptor. On read, the blob identifier is decoded and the payload is retrieved using `H5HG_read`. On delete or overwrite, `H5HG_remove` is invoked to free the corresponding global heap object.

Importantly, global heap objects are immutable in size; overwrites always allocate a new object. If a dataset element is overwritten, a new global heap object is allocated for the new payload, the descriptor is updated to reference the new heap ID, and the previous heap object is removed. Throughout this process, the dataset’s raw storage remains fixed in layout and size; only the contents of the descriptor change. This confirms that, in the current native implementation, VL payload bytes for both contiguous and chunked datasets reside in global heap objects, while dataset storage contains only fixed-size descriptors referencing those heap entries.

3.5.2 Create / Define the VL Datatype

A VL datatype is implemented as a derived datatype that wraps a base, “parent” datatype. Internally, derived types record their relationship to the underlying base type via `dt->shared->parent`. The library’s supertype query API (`H5Tget_super`) calls the internal helper `H5T_get_super`, which explicitly requires `dt->shared->parent` to be set and returns a copy of it. In other words, VL types (like other derived types) are formally modeled as datatypes that extend an underlying base type.

The behavior of VL datatypes is defined by metadata stored within the datatype’s shared representation, specifically in `dt->shared->u.vlen`, which is of type `H5T_vlen_t`. This structure captures both how VL data is interpreted and how operations on VL values are performed.

Conceptually, the VL datatype structure can be represented as:

```

/* A VL datatype */

typedef struct H5T_vlen_t {
    H5T_vlen_type_t type; /* Type of VL data in buffer */
    H5T_loc_t loc; /* Location of VL data in buffer */
    H5T_cset_t cset; /* For VL string: character set */
    H5T_str_t pad; /* For VL string: space or null padding of
                   * extra bytes */
    H5VL_object_t *file; /* File object (if VL data is on disk) */
    const H5T_vlen_class_t *cls; /* Pointer to VL class callbacks */

```

```
} H5T_vlen_t;
```

At the time of writing, the existing internal structure documentation is limited, so the following field interpretations are based on code inspection: The type field distinguishes between VL sequences and strings, which determines how payload data is interpreted and how conversions are performed. The loc field indicates whether the VL value is being handled in memory or in its on-disk representation, which affects both descriptor format and callback selection. The cset and pad fields apply only to VL strings and preserve string-specific metadata such as character encoding and padding behavior.

The file field provides the file or VOL connector context required for storage operations when the VL datatype is used on disk. The cls field is a pointer to a table of callback functions (H5T_vlen_class_t) that implement all VL-specific operations, including reading, writing, and deletion of payload data.

The callback is defined as:

```
/* VL datatype callbacks */

typedef struct H5T_vlen_class_t {

    H5T_vlen_getlen_func_t getlen; /* Function to get VL sequence size (in
    element units, not bytes) */

    H5T_vlen_getptr_func_t getptr; /* Function to get VL sequence pointer */

    H5T_vlen_isnull_func_t isnull; /* Function to check if VL value is NIL */

    H5T_vlen_setnull_func_t setnull; /* Function to set a VL value to NIL */

    H5T_vlen_read_func_t read; /* Function to read VL sequence into buffer */

    H5T_vlen_write_func_t write; /* Function to write VL sequence from buffer */

    H5T_vlen_delete_func_t del; /* Function to delete VL sequence */

} H5T_vlen_class_t;
```

As with H5T_vlen_t, the callback structure is only lightly documented in the source, so again, the role of each callback based on the current implementation is summarized below:

These callbacks provide the mechanism through which VL operations are executed. For example, the write callback is responsible for storing payload data in the appropriate backend and encoding the corresponding descriptor, while the read callback retrieves the payload data and reconstructs the in-memory representation. The del callback ensures that payload storage is correctly released when VL values are overwritten or reclaimed. This callback-driven design allows the datatype layer to remain independent of the underlying storage implementation.

For VL sequences, the public API H5Tvlen_create(base_type_id) ultimately constructs an internal datatype whose class is H5T_VLEN and whose base element type is stored in dt->shared->parent (a full copy of the provided base datatype). The VL subtype is recorded in dt->shared->u.vlen.type, and the datatype is initially configured for memory representation.

The VL-sequence constructor sets:

- `dt->shared->type = H5T_VLEN`
- `dt->shared->parent = H5T_copy(base, H5T_COPY_ALL)`
- `dt->shared->u.vlen.type = H5T_VLEN_SEQUENCE`
- `dt->shared->force_conv = TRUE` (ensures conversions perform deep copy semantics rather than aliasing)
- initial location via `H5T_set_loc(dt, NULL, H5T_LOC_MEMORY)`

This means that creating the datatype only establishes representation rules (descriptor format, conversion behavior, base-type linkage). It does not allocate any payload storage.

VL strings are created by converting a fixed-length string datatype into a VL datatype by setting its size to `H5T_VARIABLE`. Internally, this transformation occurs in `H5T__set_size` under the `H5T_STRING` case. When `size == H5T_VARIABLE`, the datatype is reclassified from `H5T_STRING` to `H5T_VLEN`, and the library assigns a parent type and VL-String metadata.

Specifically:

- The datatype is changed to `dt->shared->type = H5T_VLEN`
- The VL subtype is set to `dt->shared->u.vlen.type = H5T_VLEN_STRING`
- Character set and padding are preserved by copying them out of the string atomic fields and into `dt->shared->u.vlen.cset / dt->shared->u.vlen.pad`
- A parent type is assigned (the implementation uses a copy of unsigned char as the base element type)
- The datatype is set to memory location via `H5T_set_loc(dt, NULL, H5T_LOC_MEMORY)`

A key takeaway here is that a VL datatype's descriptor size and access semantics depend on whether the datatype is marked as "in memory" or "on disk". The library centralizes this behavior in `H5T_set_loc`, which recursively walks complex types (arrays, compounds, VL) and then delegates VL-specific handling to `H5T__vlen_set_loc(dt, file, loc)` when `dt->shared->type == H5T_VLEN`. This mechanism is also how nested VL inside compound/array types is handled: `H5T_set_loc` descends into `dt->shared->parent` when appropriate, then marks the VL wrapper itself.

As a result, "VL-ness" is not expressed by resizing dataset elements dynamically; instead, the dataset always stores a fixed-size descriptor, whose size is determined by `H5T__vlen_set_loc` for the disk location (and by the native in-memory descriptor type for memory location). The actual variable-sized payload allocation and placement occurs later during I/O (via VL conversion and the VOL blob interface), not during datatype definition.

3.5.3 Write Path: Memory -> File

A dataset write begins at the public API `H5Dwrite()`, which routes the request through the VOL layer and into the active connector's dataset write callback. For the native VOL connector, the call chain is:

```
H5Dwrite
-> H5D__write_api_common
  -> H5VL_dataset_write_direct
    -> H5VL__dataset_write
      -> cls->dataset_cls.write
        -> H5VL__native_dataset_write
          -> H5VL__native_dataset_io_setup
            -> H5CX_set_dxpl(dxpl_id)
              -> H5D__write(count, dinfo)
```

The native connector's `H5VL__native_dataset_write()` prepares per-dataset I/O state (datatypes, dataspace, buffers), installs the DXPL into the thread-local API context (`H5CX_set_dxpl(dxpl_id)`), and then invokes the internal write engine `H5D__write()`. Inside of which, HDF5 initializes datatype conversion state via `H5D__typeinfo_init()`, which first finds the conversion path via `H5T_path_find()`, then ensures VL datatypes have the correct file/VOL context attached so that later VL disk operations can call into the VOL “blob” layer:

```
/* Patch the top level file pointer for dt->shared->u.vlen.f if needed */
if (H5T_patch_vlen_file(dset->shared->type, H5F_VOL_OBJ(dset->oloc.file)) < 0)
    HGOTO_ERROR(...);
```

When type conversion is involved (which is typical for VL storage because the file representation differs from the in-memory representation), `H5D__write()` may take the selection I/O + scatter/gather pathway. In that pathway, `H5D__scatgath_write_select()` gathers the selected elements into a type conversion buffer, performs the conversion via `H5T_convert()`, and finally writes the converted file-form bytes into dataset storage with `H5F_shared_select_write()`.

For conversions where the source and destination are both VL datatypes (memory <-> disk, disk <-> memory, or VL <-> VL with different base types), the type system dispatches to the library conversion function `H5T__conv_vlen()`. This function is the concrete per-element implementation of the VL write algorithm: it reads each source VL element, optionally converts the base elements, writes the out-of-line payload to the destination storage backend, and stores the fixed-size descriptor into the destination buffer.

At conversion time (`cdata->command == H5T_CONV_CONV`), `H5T__conv_vlen()` determines whether the destination is “file-backed” by checking whether `dst->shared->u.vlen.file` is non-NULL. If so, it treats

the destination as an on-disk VL representation and uses the destination VL “class” callbacks (`dst->shared->u.vlen.cls`) to write the payload and encode the descriptor:

- `getlen` obtains the sequence length from the source descriptor.
- `read` or `getptr` obtains the source payload bytes.
- `H5T_convert` may run on the base types (the VL parent datatype) if the base conversion is not a no-op.
- `write` stores the payload out-of-line and updates the destination descriptor in place.

The critical handoff that performs the disk write is shown here:

```
/* Write sequence to destination location */
if ((* (dst->shared->u.vlen.cls->write)) (dst->shared->u.vlen.file, &vl_alloc_info, d,
                                         conv_buf, b, seq_len, dst_base_size) < 0)
    HGOTO_ERROR(...);
```

For the native VOL connector, when the VL datatype has been set to disk location (`H5T_LOC_DISK`), this write callback is `H5T__vlen_disk_write()`, which encodes the 4-byte sequence length and then delegates payload storage to the VOL blob interface, `H5VL_blob_put(...)`. The blob ID stored inline becomes the on-disk “pointer” to the global heap payload under the native connector.

Even though `H5T_conv_vlen()` sets `cdata->need_bkg = H5T_BKG_NO` during conversion-path initialization, the dataset I/O later still forces background preservation when writing into a dataset whose destination datatype contains VL. This is done in `H5D__typeinfo_init()`:

```
/* Check if we need a background buffer */
if ((io_info->op_type == H5D_IO_OP_WRITE) && H5T_detect_class(dset->shared->type,
H5T_VLEN, FALSE))
    type_info->need_bkg = H5T_BKG_YES;
```

This is necessary because overwriting a VL element requires access to the previous destination descriptor so the old out-of-line payload can be deleted. In the selection I/O pathway, this causes `H5D__scatgath_write_select()` to read the current destination bytes into the background buffer before conversion. That background descriptor is passed down into `H5T__conv_vlen()` as the `bkg` argument.

Deletion/reclamation happens in two places:

1. General overwrite handling via `setnull(..., b) / write(..., b, ...)`: both the disk `setnull` and disk write callbacks accept a background pointer (`_bg / bg`). In the disk implementation provided above, `H5T__vlen_disk_write()` deletes old payload if `bg != NULL` before storing the new payload and descriptor. This is the main “overwrite frees old heap object” behavior for non-nested cases.
2. Nested VL cleanup for shrink cases: If the VL base type itself contains VL (nested VL), `H5T__conv_vlen()` explicitly frees “leftover” deeper-level heap objects when the new sequence is shorter than the old one. This is done by iterating over the trailing elements of the old sequence (read into `tmp_buf`) and calling the destination `del` callback outlined below:

```

if (nested && seq_len < bg_seq_len) {
    uint8_t *tmp;
    size_t   u;

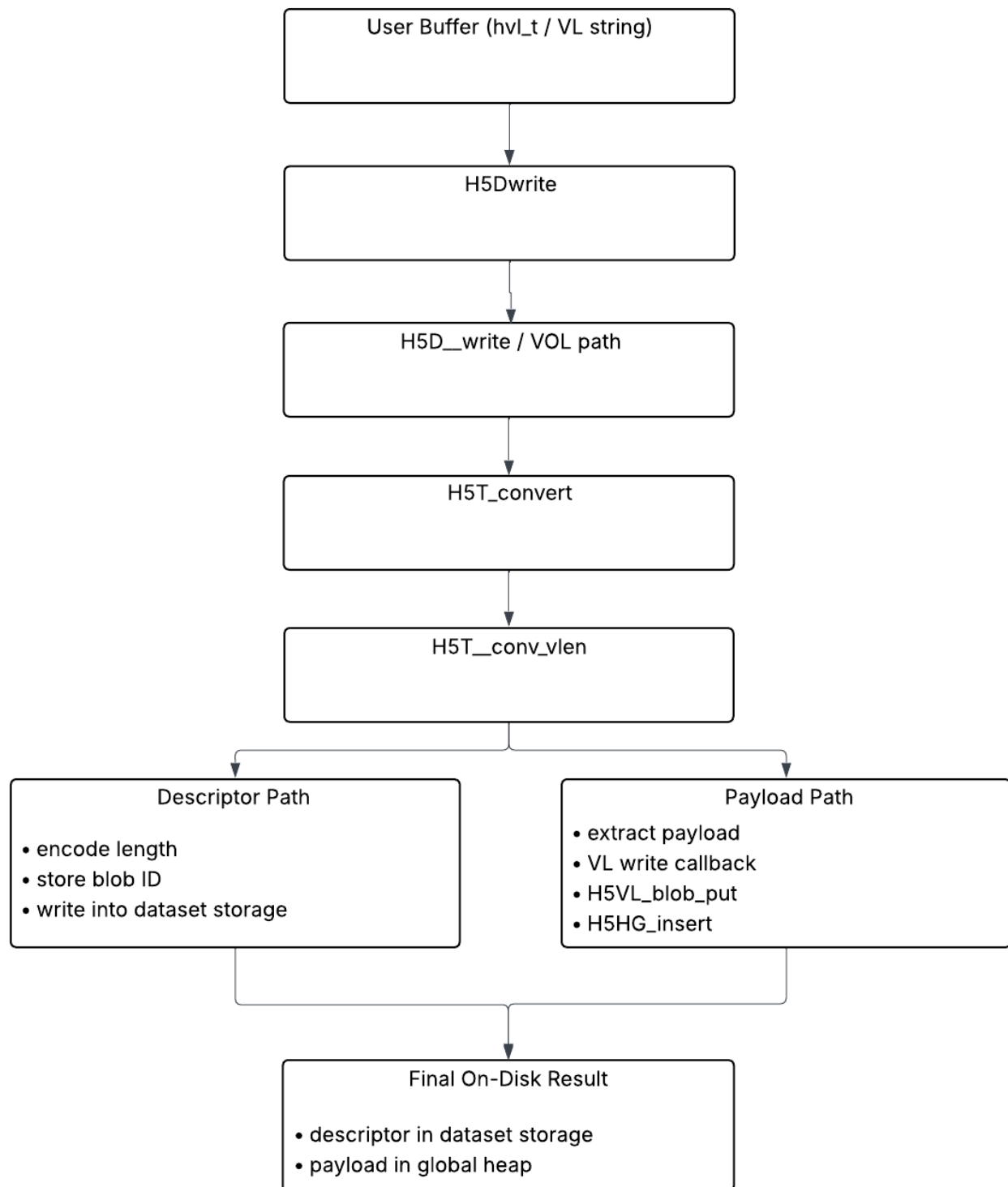
    /* Sanity check */
    assert(write_to_file);
    tmp = (uint8_t *)tmp_buf + seq_len * dst_base_size;
    for (u = seq_len; u < bg_seq_len; u++, tmp += dst_base_size) {
        /* Delete sequence in destination location */
        if ((*dst->shared->u.vlen.cls->del)(dst->shared->u.vlen.file, tmp) < 0)
            HGOTO_ERROR(...);
    } /* end for */
}

```

Together, these mechanisms explain why VL dataset writes cannot be treated as “just overwrite fixed-size bytes”: the fixed-size descriptor can be overwritten in-place, but the payload is out-of-line and must be explicitly managed (allocate new payload + delete old payload), which is why the existing destination descriptor must be available during conversion.

The diagram below summarizes the key stages of the VL write lifecycle. It highlights the separation between descriptor handling, which remains inline with dataset storage and payload handling, which is delegated to out-of-line storage through the VOL blob interface and ultimately the global heap. This separation is fundamental to the current implementation and is the primary source of indirection in VL data access.

VL Write Lifecycle (Memory->file)



Note:

If overwriting, the old payload is deleted using the existing destination descriptor before or during installation of the new payload.

3.5.4 On-Disk Storage and Caching

In the native VOL implementation, VL storage separates descriptor locality from payload locality. Dataset raw storage contains only descriptors, while payload bytes reside in global heap collections managed through the metadata cache. As a result, reading or writing VL values involves two distinct I/O domains, raw dataset I/O for the inline descriptors (chunk data for chunked layout, contiguous raw data for contiguous layout), and global heap operations for the out-of-line payloads. This separation is structural: chunk caching can improve reuse of descriptor bytes, but it does not automatically provide locality for the payload bytes referenced by those descriptors.

The global heap layer is integrated with the metadata cache. Heap collections are accessed by protecting a cache entry and later unprotecting it under the cache's global-heap entry type (H5AC_GHEAP). For example, inserting VL payload data allocates space in an existing heap collection (or creates a new one), modifies the heap buffer in memory, and then unprotects it while marking the entry dirty:

```
/* H5HG_insert: modify heap collection under metadata cache protection */
Heap = H5HG__protect(f, addr, H5AC__NO_FLAGS_SET);
/* ... allocate object slot, copy payload bytes into heap buffer ... */
heap_flags |= H5AC__DIRTIED_FLAG;
H5AC_unprotect(f, H5AC_GHEAP, heap->addr, heap, heap_flags);
```

Similarly, reading a VL payload uses read-only protection of the heap collection, copies the object's bytes into the caller's buffer, and then unprotects the collection:

```
/* H5HG_read: access heap collection via metadata cache */
heap = H5HG__protect(f, hobj->addr, H5AC__READ_ONLY_FLAG);
/* ... locate object bytes and memcpy out .. */
H5AC_unprotect(f, H5AC_GHEAP, hobj->addr, heap, H5AC__NO_FLAGS_SET);
```

Because heap collections participate in the metadata cache, VL-heavy workloads can create significant metadata-cache pressure. Each VL element read may require at least one heap-collection protect/unprotect cycle (unless multiple references hit the same collection while it is resident) and writes similarly dirty heap entries. Deletion/removal also occurs through the cache: removing a heap object dirties the collection and may mark it deleted and eligible to return space to the file free when the collection becomes empty (H5AC__DELETED_FLAG | H5AC__FREE_FILE_SPACE_FLAG).

For chunked datasets, the caching implications are especially visible. Chunk reads bring descriptor bytes into the chunk cache, but payload retrieval still requires global heap accesses. This means a sequential scan over chunked VL data often exhibits a two-stage pattern: efficient chunk I/O for descriptors followed by additional, potentially non-contiguous heap traffic to fetch payloads. The heap subsystem does attempt to improve and reuse by maintaining a CWFS (collections with free space) list of candidate heap collections with free space and advancing heaps on access (H5F_cwfs_find_free_heap, H5F_cwfs_advance_heap), but payload locality is governed by heap-allocation behavior rather than by chunk layout.

3.5.5 Read Path: File -> Memory

Reading VL data via H5Dread follows the same high-level path as writes through the VOL and dataset I/O layers, but the direction of the datatype conversion is reversed. For the native VOL connector, the call chain is:

```
H5Dread
-> H5D__read_api_common
    -> H5VL_dataset_read_direct
        -> H5VL__dataset_read
            -> H5VL__native_dataset_read
                -> H5D__read
                    -> H5D__typeinfo_init (read mode)
                        -> H5T_path_find
                            -> H5T_convert
                                -> H5T__conv_vlen (for VL types)
```

As in the write path, the critical work for VL types occurs inside the datatype conversion layer, specifically in H5T__conv_vlen, but now operating in the file -> memory direction.

At the storage level, the dataset raw data contains only fixed-size VL descriptors (length + blob identifier). When the dataset layer reads raw storage into the conversion buffer, these descriptors are passed to the VL conversion routine. For each element, the conversion code first checks whether the value is NULL. If not, it retrieves the encoded sequence length and then invokes the VL class read callback to resolve the descriptor into actual payload bytes. Under the native VOL connector, this ultimately resolves the blob identifier and reads the referenced global heap object using H5HG_read. Thus, each VL element read involves dereferencing a heap object and copying its payload into a temporary conversion buffer.

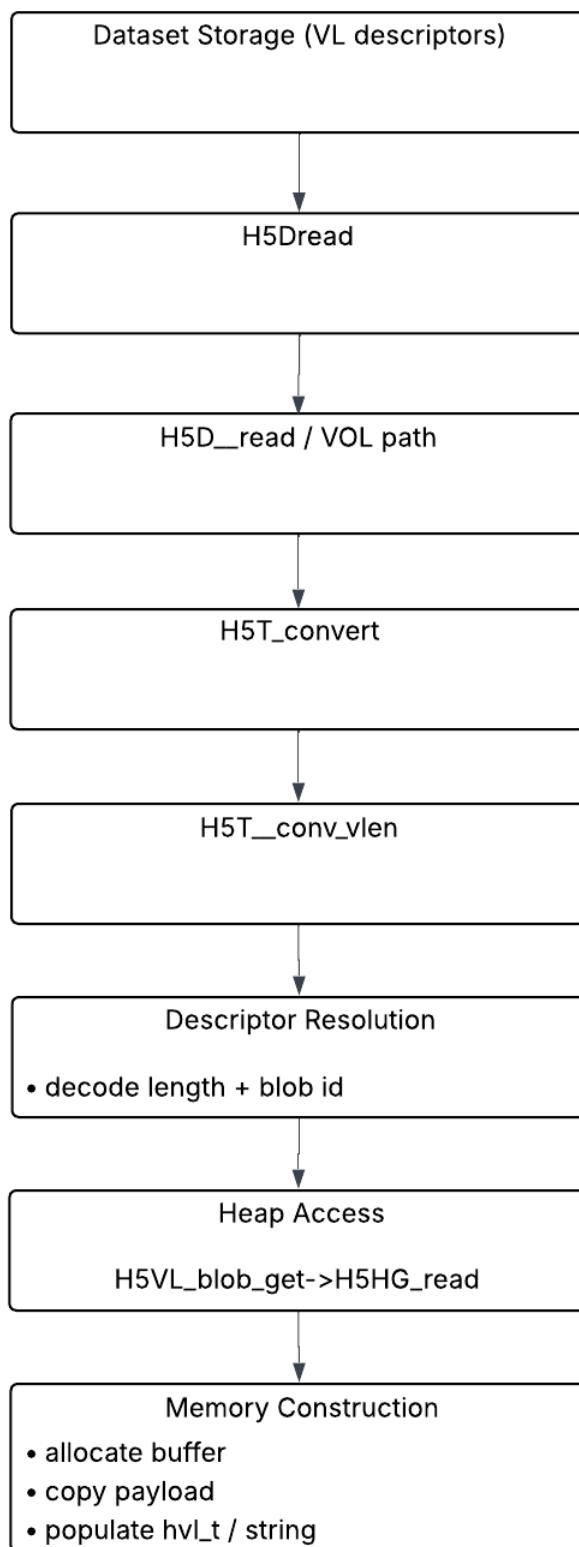
Once the payload bytes are available, base-type conversion is applied if needed. If the VL's base datatype differs between file and memory representations, H5T_convert is invoked on the sequence elements. This step may involve temporary buffers and background buffers, particularly for nested or compound types.

After conversion, the library allocates application-visible memory to hold the final VL payload. The allocation policy is determined by the dataset transfer property list (DXPL). If the user has not configured a custom allocator, the default malloc/free pair is used. Otherwise, the allocation and deallocation routines set via `H5Pset_vlen_mem_manager` are honored. The VL class write callback for memory types performs the actual allocation, copies the converted payload into the allocated buffer, and populates the user's descriptor (e.g. `hvl_t`) with the pointer and sequence length.

For nested VL datatypes (such as a VL of compound types containing additional VL members), this process recurses. Reading a single top-level element may therefore trigger multiple heap reads, conversion steps, and memory allocations at different nesting levels.

The VL read process is summarized in the diagram below, which illustrates how a VL descriptor is resolved into an out-of-line payload and reconstructed into application-visible memory.

VL Read Lifecycle (File->memory)



The diagram highlights the key stages of the VL read lifecycle. In contrast to the write path, which splits descriptor and payload storage, the read path resolves each descriptor by retrieving the corresponding payload from the global heap and constructing the in-memory representation. This indirection between dataset storage and payload retrieval is a defining characteristic of the current VL implementation.

To summarize, the VL read path consists of reading fixed-size descriptors from dataset storage, resolving heap references to retrieve payload bytes, performing base-type conversion as needed, and dynamically allocating and populating application-visible buffers. Unlike fixed-size datatypes, VL reads combine raw data I/O with metadata-managed heap access and dynamic memory construction, which explains their distinct performance and memory behavior characteristics.

3.5.6 Memory Management and Reclaim

Because VL reads allocated memory dynamically, callers cannot safely release VL data with a simple `free()` unless they can guarantee both the allocator used and the location of every nested allocation. HDF5 therefore provides `H5Dvlen_reclaim()` to reclaim memory allocated for VL data in a buffer. At a high level, this routine walks the datatype graph rooted at the memory datatype, locates VL descriptors within the application buffer, and frees each payload using the same allocation policy that was used during the read.

The allocation and deallocation policy is controlled through the dataset transfer property list (DXPL). By default, HDF5 uses the system `malloc` and `free` routines. If the application instead configures custom memory management with `H5Pset_vlen_mem_manager`, the library records those callbacks and uses them both when allocating VL payloads during `H5Dread` and when reclaiming them later. This pairing is important because freeing VL payloads with the wrong allocator would be unsafe.

At the implementation level, reclaim is handled recursively. The internal routine `H5T__vlen_reclaim` examines the datatype class of the current element and descends into arrays, compounds, and VL types as needed. For compound types, it iterates over members and recurses on complex fields. For array types, it iterates over the base elements and recurses on each one. For VL sequence types, it first recurses on the sequence contents if the base type is itself complex and then frees the sequence buffer. For VL strings, it frees the string pointer directly. This recursive structure is what allows reclaim to work correctly for nested VL datatypes and VL fields embedded inside compound or array types.

The reclaim logic uses the same allocation information structure (`H5T_vlen_alloc_info_t`) that is obtained from the API context during I/O. The public reclaim path therefore depends on the same allocator pairing that the read path used when constructing the returned in-memory representation. In this way, HDF5 ensures that VL payloads allocated during read or conversion can be released correctly and without leaks, even when the memory layout contains multiple levels of nesting.

In this document, `H5Dvlen_reclaim` refers to the public API used to reclaim VL buffers returned by `H5Dread`. Internally, the reclaim traversal is performed by datatype code such as `H5T__vlen_reclaim` and covers VL sequences, VL strings, and nested VL fields according to the memory datatype. This is distinct from freeing arbitrary application-owned strings; `H5Dvlen_reclaim` is required when the library allocated VL memory as part of an HDF5 read/conversion result.

3.5.7 Delete / Overwrite / Object Lifetime

The lifetime of a VL payload is separate from the lifetime of the descriptor stored in dataset raw storage. The descriptor is fixed-size and remains part of the dataset element layout, while the payload it references may be created, replaced, or deleted over time. As a result, overwrite and deletion operations must manage both the descriptor and the out-of-line heap object it refers to.

For ordinary writes of new VL data, the library allocates a new heap object for the payload object and encodes the resulting blob identifier into the destination descriptor. For overwrites, the previous payload must also be removed. This is why the dataset I/O layer forces a background buffer for VL writes: the existing destination descriptor must be available during conversion so the old heap object can be identified and deleted. In the native disk implementation, `H5T__vlen_disk_write` accepts the previous descriptor through its background argument and removes the old payload before or during installation of the new one. Similarly, `H5T__vlen_disk_setnull` deletes any previously referenced payload when converting a VL value to the nil state.

Nested VL datatypes introduce an additional lifetime issue. If the new sequence is shorter than the previous one, simply replacing the top-level descriptor is not sufficient, because deeper-level heap objects corresponding to the truncated tail would otherwise remain allocated. The conversion routine `H5T__conv_vlen` therefore performs explicit cleanup for nested shrink cases by iterating over the trailing elements of the old sequence and invoking the destination VL deleted callback (`del`) on each one. This ensures that no deeper heap objects are orphaned when nested sequences contract.

At the heap layer, deletion is handled by `H5HG_remove`, which removes the referenced object from its heap collection. If the collection becomes empty, the heap entry may be marked deleted and returned to file free space. This means VL overwrite and delete behavior is not just a matter of updating dataset bytes; it also involves explicit heap object lifetime management. The descriptor and payload therefore have related but distinct lifecycles: the descriptor persists as part of the dataset layout, while the payload object may be repeatedly replaced or destroyed as values change.

3.5.8 SWMR Visibility Limitations

The current global-heap-based VL implementation introduces visibility and consistency challenges for SWMR-style access because the fixed-size descriptor and the payload it references are not stored together. The descriptor lives in dataset raw storage, while the payload lives separately in a global heap collection managed through the metadata cache. As a result, the logical value of a single VL element is split across two different storage domains that are updated and cached separately.

This separation matters because a reader that observes an updated dataset descriptor may still require separate access to the corresponding heap object to reconstruct the complete VL value. Unlike fixed-size dataset elements, where the value is fully contained within the dataset's raw storage, a VL element depends on additional metadata-managed state outside the dataset or chunk itself. For chunked datasets especially, this means chunk-local visibility is not enough to guarantee value-local visibility:

seeing the updated chunk data does not by itself imply that the associated payload bytes are chunk-local or naturally synchronized with that chunk.

This behavior is a consequence of how updates and visibility are coordinated between dataset storage and heap storage, rather than a fundamental correctness issue. When a VL value is written or modified, both the descriptor in the dataset and the corresponding heap object must become visible to readers in a consistent manner. This typically requires coordinated flushing or ordering of operations so that a reader does not observe a descriptor that references a payload that is not yet visible, or a payload that is no longer referenced by any descriptor.

Because these updates involve separate storage domains (dataset raw data and global heap metadata), ensuring this consistency requires additional coordination compared to fixed-size datatypes, where the entire value is contained within a single storage location. While this coordination can be achieved, it introduces additional complexity and may impact performance depending on how synchronization is implemented.

This highlights that the global-heap-based design is not inherently incorrect, but that it is less naturally aligned with chunk-oriented consistency models. Because the payload is not physically co-located with the chunk, its visibility and lifetime are mediated through the global heap subsystem rather than through chunk-level mechanisms. This architectural separation is one of the motivations for exploring chunk-local VL storage designs, which aim to improve locality and simplify consistency by keeping descriptor and payload data together.

4 Global Heap (H5HG) Analysis

The previous section describes how VL data is represented and processed at the datatype and I/O levels. This section analyzes the global heap implementation (H5HG) with the goal of identifying which behaviors can be reused and which must be adapted for a chunk-local storage model.

The goal here is to examine how heap collections are structured, how space is allocated and reused, and how payload objects are created, accessed, and removed over time. This includes understanding how heap operations interact with the metadata cache and how heap state evolves as VL data is written, overwritten, and deleted.

This analysis is important because the global heap is the primary storage mechanism for VL payloads in the current implementation. Any redesign that aims to localize VL data within dataset chunks must either replace or adapt this behavior. In particular, the chunk-local heap approach described in Section 5.2 proposes using portions of the existing global heap logic in a different storage context. A detailed understanding of H5HG is therefore necessary to determine which aspects of the current implementation can be preserved and which must be modified.

The following subsections describe the structure and behavior of the global heap and identify key properties that influence potential redesigns.

4.1 Heap Structure and Layout

The global heap is implemented as a collection-based storage system managed through the H5HG interface. Each heap collection is represented in memory by an H5HG_heap_t structure and corresponds to a contiguous region of storage in the file. In this section, ‘contiguous’ refers to the storage occupied by a single heap collection, not to the global heap as a whole and not to contiguous dataset layout. New heap collections are created via H5HG_create, which allocates file space, initializes the heap header, allocates the in-memory collection buffer (heap->chunk), and sets up the array of heap objects (heap->obj). The heap is then inserted into the metadata cache as an H5AC_GHEAP entry. In the current implementation, global heap collections store VL payload data but are managed through the metadata cache rather than through raw dataset I/O paths. This distinction matters because H5HG manages a set of heap collections, each treated as its own I/O caching unit, rather than a single monolithic heap.

The in-memory heap collection structure (H5HG_heap_t) can be represented as:

```
struct H5HG_heap_t {
    H5AC_info_t cache_info;    /* Information for H5AC cache functions, MUST be
                               * the first field in structure
                               */

    haddr_t  addr;             /* Collection address */

    size_t   size;             /* Total size of collection */

    uint8_t *chunk;           /* Collection of elements - note that this
                               * INCLUDES the header, so it's not just
                               * the objects!
                               */

    size_t nalloc;             /* # object slots allocated */

    size_t nused;              /* # of slots used
                               * If this value is >65535 then all indices
                               * have been used at some time and the
                               * correct new index should be searched for
                               */

    struct H5F_shared_t *shared; /* Shared file */

    H5HG_obj_t *obj;           /* Array of object descriptions */
}
```

```
};
```

The source-level documentation for this internal structure is limited, so the fields are interpreted based on code inspection:

The `H5HG_heap_t` structure represents the in-memory state of a single global heap collection. Conceptually, it encapsulates three key components: a contiguous heap buffer (`chunk`), an object table (`obj`), and metadata describing the heap's size and location. These components together define how variable-length payloads are stored and accessed.

The `chunk` field stores the raw bytes of the heap collection, including both the heap header and all heap objects. This shows that heap objects are not independently allocated in file space, but instead packed into a single contiguous buffer. The `obj` array serves as an object table, where each entry tracks the location and size of a heap object within this buffer. Together, these two fields define how variable-length payloads are physically organized and accessed.

The `addr` field identifies the heap collection within the file, and when combined with an object index (stored separately in heap object handles), forms the (`addr`, `idx`) pair used to uniquely identify a VL payload. This mechanism is central to how descriptors reference heap-stored data.

The `size`, `nalloc`, and `nused` fields track the overall capacity and usage of the heap, including how many object table entries are allocated and in use. In particular, the use of `nused` in combination with the object table reflects the design choice that heap objects are managed through indexed entries rather than direct pointers.

Finally, the `cache_info` field indicates that heap collections are managed as metadata cache entries. This reinforces that global heap storage is tightly coupled to the metadata cache system, with heap collections being loaded, modified, and flushed through cache mechanisms rather than accessed directly through dataset I/O paths.

Within a collection, payloads are represented as heap objects tracked through the `heap->obj` array. Each object entry records metadata including its size, reference count, and starting location within the heap buffer. The actual object bytes are stored inside the contiguous heap buffer (`heap->chunk`), and the `begin` field of each object points into that buffer. Heap objects are therefore not allocated independently from the file. Instead, they are packed into a single cache-managed collection buffer and located relative to that in-memory chunk of heap storage.

Object identity is based on the heap collection address together with an object index. This is visible in `H5HG_insert`, which returns a heap object handle by filling `hobj->addr` with the heap collection address and `hobj->idx` with the allocated object index. Later accesses use this pair to reload the collection and locate the object within the `heap->obj`. `H5HG_read`, for example, protects the heap collection using the stored address and then uses `hobj->idx` to retrieve the object's size and starting pointer.

Free space is represented explicitly as object index 0. During heap creation, `H5HG__create` initializes `heap->nused = 1` to reserve index 0 for the free-space object and then writes a free-space object header into the heap buffer. Allocation logic in `H5HG__alloc` also treats index 0 specially, first asserting that `heap->obj[0].size` is large enough for the requested allocation and then carving the new object out of

the free-space region beginning at `heap->obj[0].begin`. As a result, free space is not tracked as an arbitrary set of holes; it is managed through a designated free-space entry inside the heap object table.

Heap collections are managed as metadata cache entries. This is reflected in the declaration of `H5AC_GHEAP` as a metadata entry class and in the metadata cache client structure, which includes callbacks such as `deserialize`, `pre_serialize`, `serialize`, and `free_icr`. In practice, access to a heap collection is mediated through `H5HG__protect`, which wraps `H5AC__protect` and returns an in-memory `H5HG_heap_t` for the requested file address. Modifying operations such as `H5HG_insert`, `H5HG__alloc`, `H5HG_extend`, and `H5HG_remove` update the heap in memory and then unprotect it with appropriate cache flags, typically including `H5AC__DIRTIED_FLAG` when the heap contents have changed.

Heap collections can also grow. `H5HG_extend` reallocates the heap buffer, updates the recorded heap size, adjusts the object pointers so they continue to point into the resized buffer, and expands the free-space object accordingly. This is consistent with the observation that the existing global heap already appears to support extension and may therefore be reusable as the basis for a chunk-local heap, provided it can be adapted to operate in a chunk-relative context.

Overall, each global heap collection is defined by three core elements: a contiguous heap buffer, an object table describing live and free regions within that buffer, and a cache-managed collection address that allows the collection to be reloaded and manipulated through the metadata cache. This structure is important because the chunk-local approach proposal is based on reusing as much of this machinery as possible rather than introducing a completely new heap implementation.

4.2 Allocation and Free Space Management

Allocation of VL payloads in the global heap is performed through `H5HG_insert`, which coordinates selection of a heap collection and placement of a new object within that collection. The allocation process operates at two levels: first selecting an appropriate heap collection and then allocating space within that collection.

At the collection level, `H5HG_insert` attempts to reuse an existing heap collection with sufficient free space before creating a new one. This is handled through calls such as `H5F_cwfs_find_free_heap()`, which searches for a collection that can satisfy the allocation request. If no suitable collection is found, the implementation may attempt to extend an existing collection via `H5HG_extend` or ultimately create a new collection using `H5HG__create`. This behavior ensures that heap storage is reused when possible and that new collections are only created when necessary.

Once a heap collection is selected, allocation within the collection is handled by `H5HG__alloc`. This function operates on the designated free-space object stored at `heap->obj[0]`. It first verifies that the free-space region is large enough to satisfy the requested allocation. The new object is then carved out of this region by assigning its begin pointer to the current begin pointer of the free-space object and reducing the free-space object accordingly. The free-space begin pointer is advanced past the allocated region, and a new entry is added to the `heap->obj` array to describe the allocated object.

This shows that allocation proceeds in a linear, region-based manner within each heap collection. More precisely, allocation is carved from the designated free-space object (`heap->obj[0]`), and the actual

space consumed includes both the payload and the heap object header, rounded for alignment. Rather than searching for arbitrary holes, the system maintains a single contiguous free region and is allocated from it directly. This design simplifies allocation logic and avoids fragmentation within the active portion of the heap, except for small residual regions that cannot accommodate a new object header.

Reuse of space is driven by deletion and compaction. When objects are removed via `H5HG_remove`, the heap may shift subsequent objects to eliminate gaps and merges the freed space into the free-space object. As a result, space reclaimed from deletions becomes immediately available for reuse through subsequent calls to `H5HG__alloc`. Note that if the remaining free space is too small to hold another heap object header, that tail space is retained but cannot immediately describe a new object, so the implementation tolerates a small amount of internal waste. While this compaction behavior may involve moving memory within the heap buffer, its performance implications are not a primary concern for the initial implementation of a chunk-local heap model. More advanced compaction or optimization strategies can be considered as a later refinement if necessary.

Heap extension provides an additional mechanism for accommodating new allocations. When the free-space region is insufficient, `H5HG_extend` increases the size of the heap collection, adjusts internal pointers, and expands the free-space object. This allows the heap to continue allocating within the same collection without requiring immediate creation of a new one.

Overall, allocation and free-space management in the global heap are characterized by reuse of existing heap collections when possible, allocation from a single contiguous free-space region within each collection, and reclamation of space through local compaction on object removal. This model is important for understanding potential redesigns, as it demonstrates that the current implementation already enforces a compact, append-like allocation strategy within each collection, rather than maintaining fragmented free-space regions.

4.3 Removal and Compaction

Removal of heap objects is handled by `H5HG_remove`, which reclaims the storage associated with an object and updates the heap collection so that free space remains usable for later allocations. Unlike designs that leave holes in place and maintain a fragmented free-space map, the global heap performs local compaction when an object is removed.

The removal process begins by locating the target object through its heap collection address and object index. Once the object is identified, `H5HG_remove` computes the total span of storage to reclaim, including both the payload size and the heap object header, rounded for alignment. This reclaimed span is represented by the local variable `need`, which determines how much storage will be removed from the active region of the heap.

After determining the removed span, the function adjusts the positions of all later objects in the collection. For each object whose begin pointer lies after the removed object, `H5HG_remove` adjusts that pointer by subtracting the reclaimed span before performing the `memmove` that compacts the trailing bytes. The implementation then performs the actual compaction with a `memmove`, sliding the trailing portion of the heap buffer left over the deleted object's region.

This operation eliminates the gap that would otherwise be created by deletion. Rather than preserving interior holes, the heap keeps live objects packed together and moves the reclaimed space to the end of the collection. The free-space object (`heap->obj[0]`) is then updated to reflect the enlarged free region. If no free-space object currently exists, one is reinitialized at the end of the heap; otherwise, its size is increased accordingly. When enough space remains to hold a heap object header, the implementation rewrites the free-space object header in the heap buffer.

Once compaction is complete, the removed object's metadata entry is cleared with `memset`, effectively marking that object table slot as unused. The heap is then marked dirty so the updated collection will be written back through the metadata cache.

`H5HG_remove` also handles the special case where deletion leaves the collection empty. If the free-space region grows to occupy the entire heap apart from the collection header, the heap collection is treated as empty and is marked for deletion and return to file free space. Otherwise, the collection remains active and may be advanced within the CWFS (collection-wide free space) tracking structure so it can continue to participate in future allocation decisions.

This behavior is important because it shows that the current global heap implementation already includes a built-in mechanism for reclaiming space without leaving fragmented interior gaps. Space reuse therefore occurs by compacting the heap locally and enlarging the free-space region, which can then be used by subsequent allocations. This is directly relevant to the chunk-local heap proposal since it suggests that at least part of the required compaction behavior may already exist in reusable form.

4.4 Extension and Growth

Heap collections are not fixed in size and can be expanded when existing free space is insufficient to satisfy an allocation request. This behavior is implemented in `H5HG_extend`, which increases the size of a heap collection and updates its internal state to accommodate additional payloads.

Extension is typically triggered when allocation logic in `H5HG_insert` determines that no existing heap collection contains enough free space, even after considering reuse. In such cases, rather than immediately creating a new heap collection, the implementation may choose to extend an existing one. This helps reduce the number of heap collections in use and improves locality for related payloads.

The `H5HG_extend` function operates on an in-memory `H5HG_heap_t` structure. It first reallocates the heap buffer (`heap->chunk`) to a larger size, increasing the total capacity of the collection. Because the heap buffer may be relocated during this reallocation, all object pointers stored in `heap->obj[]` must be updated to point into the new buffer location. This is done by computing the offset of each object relative to the original buffer and reassigning its begin pointer accordingly.

Once the buffer has been resized, the heap updates its internal size metadata and expands the free-space (`heap->obj[0]`) to include the newly added region. This effectively appends additional contiguous space to the existing free region, allowing future allocations to proceed without creating a new collection. As with other modifying operations, the heap is then marked dirty so that the updated state will be written back through the metadata cache.

This extension mechanism complements the allocation and compaction behavior described in earlier sections. While `H5HG_remove` reclaims space within the existing bounds of a heap collection, `H5HG_extend` increases those bounds when necessary. Together, these mechanisms allow heap collections to both reuse space efficiently and grow dynamically in response to workload demands.

From a design perspective, this behavior is particularly relevant to chunk-local heap proposals. The ability to grow a heap in place and maintain consistent object addressing suggests that the existing implementation already supports many of the behaviors required for managing a variable-sized payload region. However, in the current design, this growth is tied to file-level allocation and metadata cache management, which may need to be adapted if the heap is instead embedded within a chunk-local storage layout.

4.5 Metadata Cache Interaction

Global heap collections are not managed as standalone data structures, but rather as metadata cache entries integrated into the HDF5 metadata caching system. Each heap collection is associated with the metadata cache entry class `H5AC_GHEAP`, which defines how the heap is loaded from disk, serialized back to disk, and managed in memory.

Access to a heap collection is performed through `H5HG__protect`, which internally calls `H5AC_protect` with the `H5AC_GHEAP` class. This operation retrieves the in-memory representation of the heap (`H5HG_heap_t`) corresponding to a given file address and ensures that the collection is safely accessed under the metadata cache's protection mechanisms. Once an operation is complete, the heap is released using `H5AC_unprotect`, with flags indicating whether the collection was modified (e.g. `H5AC__DIRTIED_FLAG`).

Because heap collections are metadata cache entries, their lifecycle is governed by the cache. When a heap is first accessed, it is deserialized from disk using the cache's deserialize callback. Modifications to the heap, such as object insertion, removal, or extension, update the in-memory representation and mark the entry as dirty. The updated heap is then written back to disk through the cache's serialize callback when it is evicted or explicitly flushed.

This design means that all heap operations are performed on a cache-resident, mutable representation of the heap collection, rather than directly in disk structures. It also implies that heap state is subject to cache policies such as eviction, protection, and synchronization, and that heap modifications are coordinated with other metadata updates through the cache infrastructure.

The tight integration with the metadata cache has important implications for redesign efforts. In the current implementation, heap collections exist as independent metadata objects, separate from dataset storage and managed through a global cache. In contrast, a chunk-local heap design would require heap state to be embedded within the chunk representation and managed through the shared chunk cache callbacks used to gather, serialize, deserialize, and scatter chunk-local VL state. This suggests that while core heap behaviors such as allocation, compaction, and extension may be reusable, the surrounding lifecycle management and serialization mechanisms are likely to require adaptation.

4.6 Implications for Redesign

The analysis of the global heap implementation highlights several properties that are directly relevant to the proposed chunk-local heap approach.

First, the core heap behaviors – allocation (`H5HG__alloc`), object tracking (`heap->obj`), local compaction (`H5HG_remove`), and extension (`H5HG_extend`) – are largely self-contained within the `H5HG_heap_t` structure and operate on a contiguous in-memory buffer. These mechanisms are not inherently tied to VL datatypes themselves, but rather to how variable-sized data is managed within a region of memory. This suggests that a significant portion of the existing heap logic may be reusable in a chunk-local context, where a similar contiguous buffer is maintained within a dataset chunk.

Second, the current implementation enforces a compact layout by design. Objects are densely packed within the heap buffer, and free space is consolidated into a single region (`heap->obj[0]`). Allocation proceeds linearly from this region, and deletion restores space through compaction rather than fragmentation. This behavior aligns well with the requirements of a chunk-local heap, where maintaining a contiguous payload region is desirable for both simplicity and performance.

However, several aspects of the current implementation are tightly coupled to the metadata cache and file-level storage model. Heap collections are identified by file addresses, managed as `H5AC_GHEAP` entries, and accessed through `H5HG__protect` and `H5AC_protect`. Serialization and deserialization are handled through metadata cache callbacks, and heap state persists independently of dataset chunks. These behaviors would not directly apply in a chunk-local design, where heap data must instead be embedded within the chunk and managed through the chunk cache pipeline.

As a result, a chunk-local heap implementation would likely require separating the core heap management logic from metadata-cache-specific behavior. In particular:

- Heap storage would need to be defined relative to a chunk buffer rather than a file address.
- Serialization and deserialization would be integrated into chunk-level encode/decode operations instead of metadata cache callbacks.
- Heap lifecycle management would be driven by chunk access patterns rather than metadata cache protection and eviction.

Overall, the global heap implementation provides a strong foundation for managing variable-sized data within a contiguous region, but its integration with the metadata cache represents a key boundary that must be addressed. A successful chunk-local heap design would aim to reuse the internal allocation and compaction mechanisms while replacing the surrounding storage and lifecycle model with one that is aligned with chunk-based data management.

This separation between reusable mechanics and metadata-cache-dependent infrastructure is a key enabler for adapting the existing implementation to a chunk-local storage model.

5 Proposed Designs

This section evaluates approaches for improving the storage and access of variable-length (VL) data in HDF5. As described in Section 3, the current implementation relies on global heap storage and descriptor-based indirection, which can introduce additional I/O overhead, reduced locality, and increased pressure on the metadata cache, particularly for VL-heavy workloads.

All proposed designs described in this section operate within a common set of architectural constraints. In particular, VL data handling must integrate with the shared chunk cache pipeline, including scatter/gather and serialize/deserialize stages of chunk processing. The goal of these designs is to modify how VL payloads are stored and accessed, while preserving existing datatype semantics, including nested VL handling, recursion and conversion behavior.

These approaches are intended for chunked datasets and do not attempt to address VL storage for contiguous datasets. Additionally, all proposed designs must coexist with the current heap-based implementation to maintain compatibility with existing data. As a result, the primary area of change across all approaches is the storage and organization of VL payloads, rather than the higher-level datatype or I/O logic.

The following subsections describe each approach in detail.

5.1 Flattened Chunk-Local Model

This approach, proposed by Neil Fortner, replaces the current global-heap-based storage model with a chunk-local layout in which both fixed-size descriptors and VL payloads are stored within the dataset's chunk data.

Under this design, each chunk is logically divided into two regions:

- A fixed-length region containing dataset elements of the file datatype.
- A variable-length (VL) region containing the serialized payloads referenced by those elements.

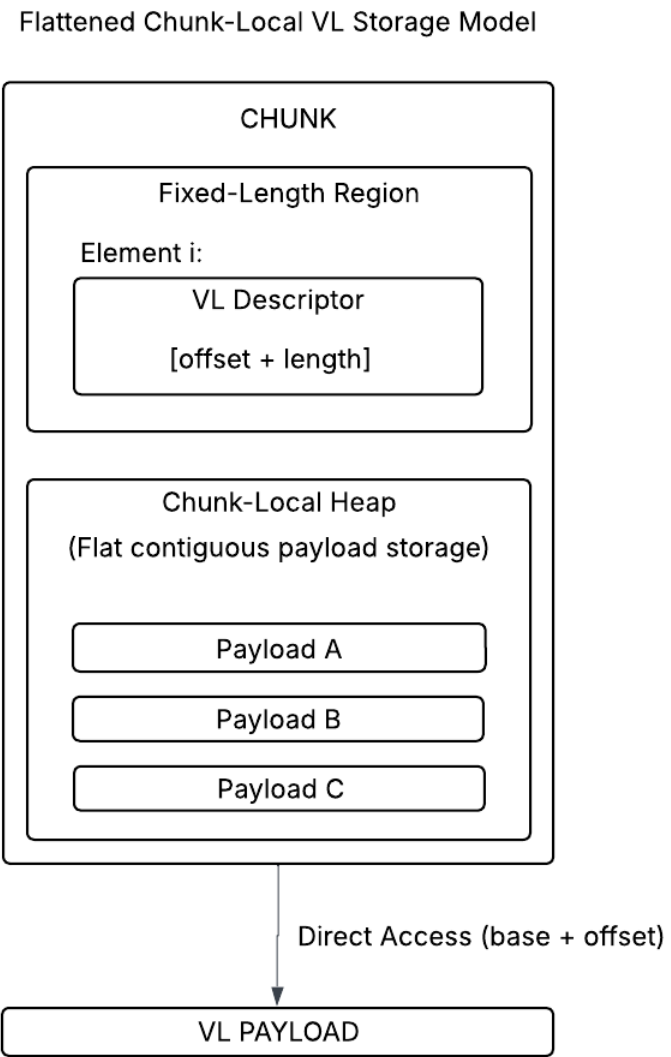
For any variable-length or reference datatype fields, the fixed-length region stores two 8-byte unsigned integers: the byte offset of the associated payload within the VL region, and the length of that payload in bytes. These values describe the location of the payload within the chunk.

The VL region contains the actual payload data, stored as a sequence of contiguous byte segments. Each payload is interpreted according to standard HDF5 datatype rules. For variable-length array, the number of elements can be derived from the payload size and the size of the base datatype.

Nested variable-length datatypes are represented using a flattened encoding. At each level of nesting, the payload begins with an 8-byte integer representing the number of elements at that level, followed by the serialized data for those elements. This pattern is applied recursively, allowing nested structures to be represented without additional indirection.

Under this layout, a dataset element is resolved by reading its descriptor and computing the payload location directly within the VL region. This replaces the need for heap lookup and removes the separation between dataset storage and payload storage by collocating both within the same chunk.

The diagram below illustrates this model. Each descriptor directly references its payload using an offset into the VL region, and the payload is retrieved without any interaction with a separate heap structure.



This design changes where VL data resides. Instead of storing payloads in external heap collections, all data associated with a chunk is contained within that chunk. As a result, VL access becomes a purely chunk-local operation, improving locality and eliminating metadata cache dependency for payload resolution.

Overall, the flattened model simplifies the storage representation by eliminating heap indirection but introduces new responsibilities for layout construction and space management within the chunk layer.

5.1.1 Deeper Implementation Analysis

While the flattened model clearly defines the on-disk representation of VL data, it does not directly specify how that representation is constructed, maintained, or updated within the existing HDF5 system. In the current implementation, these responsibilities are handled by the global heap subsystem, which provides allocation, compaction, and object tracking. In the flattened model, these responsibilities must be implemented within the chunk layer itself.

At the storage level, each chunk must now maintain both its fixed-length representation and a VL region containing all associated payloads. This requires treating the chunk as a structured object with two coordinated regions: one for descriptors and fixed-size data, and one for serialized payloads. Unlike the global heap, there is no separate object table or metadata describing payload placement beyond what is encoded in the descriptors. The offset and length stored in each descriptor become the sole mechanism for locating payload data.

This design introduces new requirements for space management within the VL region. Without a heap abstraction, the system must determine how payload space is allocated and reused. A simple approach is to treat the VL region as append-only, placing new payloads sequentially at the end of the region. While straightforward, this approach leads to fragmentation when elements are updated or removed as unused regions accumulate over time.

To avoid fragmentation without introducing complex allocation logic, a practical strategy for an initial implementation is to rebuild the VL region whenever a chunk is rewritten. Under this approach, all active payloads are collected, repacked contiguously into a new VL region, and the corresponding descriptors are updated with new offsets. This avoids incremental compaction and ensures a consistent layout at the cost of additional work during chunk writes.

Update behavior is directly impacted by this design. Because descriptors store explicit offsets, any relocation of payload data requires updating those descriptors. Unlike the global heap model, this design does not provide stable identifiers for payloads, as offsets change whenever the VL region is rebuilt. Unlike the global heap model, where object indices remain stable across compaction, the flattened model ties descriptor validity directly to payload position. This reinforces the rebuild-on-write strategy, in which all offsets are recomputed in a single pass.

This rebuild-on-write discussion applies to the flattened-offset model, where descriptors contain direct offsets into the VL region. It is not the selected policy for initial implementation with the H5HG-style chunk-local heap approach. In the selected approach, descriptors contain local heap object indices, and the heap object table provides the indirection needed to avoid recomputing every descriptor on each write. Heap compaction may still occur during removal, encode/flush, or under a later wasted-space policy, but the design does not require rebuilding the entire VL region for every update.

The read and write paths become structurally simpler but still require coordinated processing. On write, VL data is gathered, serialized into the VL region, and assigned offsets, while descriptors are updated to reflect offsets into the VL region. Although this removes global heap lookup, the conversion between serialized and in-memory VL representation remains necessary and unchanged.

Nested VL datatypes introduce additional complexity in this model. Because nested sequences are encoded recursively within the VL region, parsing requires dynamically walking the structure. At each level, the parser reads the element count and interprets the subsequent data accordingly. This matches

the semantics of the current implementation but shifts all responsibility for interpreting nested structures into the chunk-local logic.

From a systems perspective, this design integrates naturally with the chunk processing pipeline. The VL region is constructed during the gather phase, serialized alongside the fixed-length region, reconstructed during deserialization, and accessed during scatter. Because all payload data is contained within the chunk, no interaction with the metadata cache or global heap subsystem is required during these operations.

Overall, the flattened model removes heap indirection and simplifies the descriptor format, but does so by shifting allocation, compaction, and layout management responsibilities into the chunk layer. While the resulting structure is more localized and easier to reason about conceptually, it requires introducing new mechanisms that replicate functionality currently provided by the global heap.

5.1.2 Implementation Requirements and Responsibilities

The flattened chunk-local model replaces the global heap with a chunk-local encoding, requiring all responsibilities related to VL payload management to be implemented within the chunk layer. In the current system, these responsibilities are handled by the global heap subsystem, including allocation, compaction, and object tracking. Under the flattened model, these responsibilities must be redefined in terms of chunk construction and layout.

During write operations, payload placement becomes part of chunk construction. Instead of calling heap allocation routines such as `H5HG_insert` or `H5HG__alloc`, the system serializes payload data directly into the VL region and computes offsets for each associated descriptor. This shifts responsibility for layout from the heap subsystem to the chunk-building logic.

This process aligns naturally with the gather phase of the chunk processing pipeline. As elements are gathered, their associated VL data is collected and serialized into a contiguous region. Descriptor values are then populated with the corresponding offset and length values, completing the mapping between fixed-length and VL regions.

In practice, chunk construction may involve assembling multiple regions (such as fixed-length data and the VL region) independently, then combining them into a final chunk image. These regions can be written together using a vector write, after which lower-level filtering and compression are applied. This allows the VL layout logic to operate independently of storage-level concerns.

On the read path, the deserialize phase reconstructs the chunk buffer, including both the fixed-length and VL regions. Payload resolution is then performed by interpreting descriptor offsets directly, eliminating the need for external heap lookup. The scatter phase continues to reconstruct application-visible VL data using the existing datatype conversion framework.

Because payloads are stored within a contiguous VL region, updates may require reorganization of this region. For an initial implementation, this can be handled using a rebuild-on-write approach, where all active payloads are collected, repacked into a new contiguous VL region, and descriptor offsets are recomputed. This simplifies layout management without requiring incremental free-space tracking.

This is a drawback of the flattened-offset model and is one reason it is not selected as the first implementation path. The chunk-local heap model preserves a heap object table so payload movement does not automatically require rewriting every descriptor that refers to the moved bytes.

Update behavior is closely tied to layout. Because descriptors store direct offsets, modifications that change payload size require recomputing offsets during chunk reconstruction. In practice, this aligns with the rebuild-on-write model, where the chunk is reconstructed as a whole during write operations.

Nested VL datatypes require recursive interpretation within the VL region, as each level encodes its own element count and data. At each level, the parser reads the element count and interprets the subsequent data accordingly. This matches the semantics of the current implementation but shifts all responsibility for interpreting nested structures into the chunk-local logic.

Finally, the flattened model increases sensitivity to layout consistency. Because descriptors directly reference payload locations, any corruption or inconsistency in the VL region may invalidate multiple descriptors. This differs from the global heap model, where payload storage is more loosely coupled from descriptor storage.

Overall, implementing the flattened model requires introducing chunk-level mechanisms for layout construction, space management, and descriptor coordination. While this removes external indirection and simplifies the storage model conceptually, it shifts complexity into the chunk processing layer and requires careful handling of updates, layout constraints, and nested data structures.

5.2 Chunk-Local Heap Model

An alternative direction suggested by John Mainzer is to retain the existing descriptor-based representation of VL data while restructuring how heap storage is organized so that it is associated directly with individual dataset chunks.

In the current implementation, VL payloads are stored in global heap collections that are independent of dataset layout. Each dataset element contains a descriptor with a length and blob identifier, and accessing the payload requires resolving this identifier through the heap subsystem. This results in a two-step access pattern: first reading dataset storage (typically chunk data) and then performing separate metadata-driven accesses to heap storage.

Rather than storing VL payloads in file-level global heap collections that are independent of chunk layout, this approach associates heap storage directly with individual chunks.

Under this model, all VL payloads associated with elements in a given chunk are stored within that chunk's heap. The descriptor stored in the dataset element continues to contain a length and identifier, but that identifier now refers to an entry within the chunk's associated heap rather than a global heap structure.

Conceptually, this means that:

- Each chunk maintains its own heap-like storage region for VL payloads.

- VL data written to elements within a chunk is allocated from that chunk's heap.
- When a chunk is read, both its descriptors and all referenced payloads can be accessed within the same logical unit of storage.

Importantly, this design can reuse much of the existing global heap implementation. Rather than developing a new heap system from scratch, the existing global heap code can be adapted so that each chunk is treated as an independent heap instance. The heap is constructed in memory during I/O operations and serialized as part of the chunk's on-disk representation.

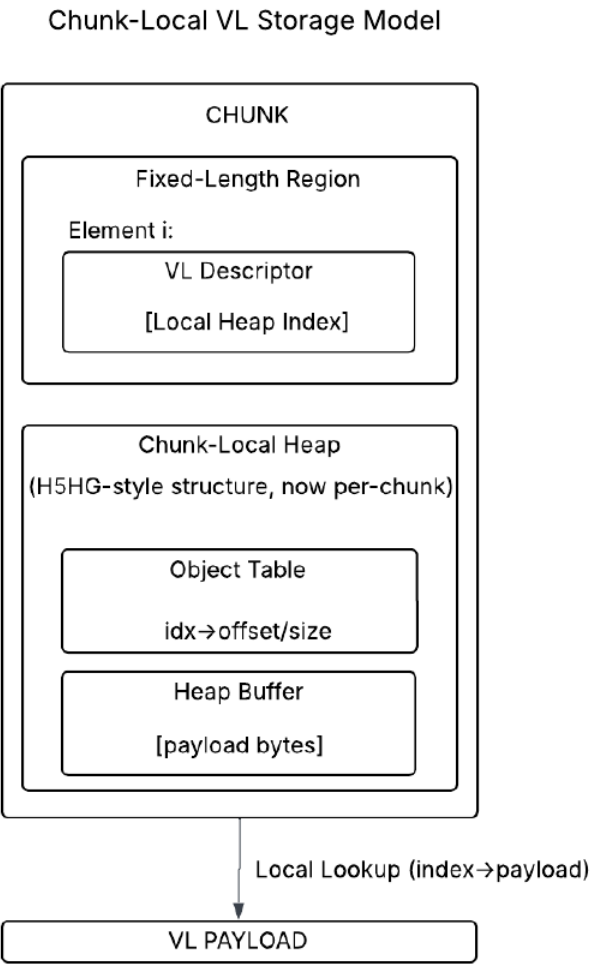
This approach potentially eliminates the need to coordinate access between dataset storage and a separate global heap, since both descriptors and payloads are now scoped to the same chunk. As a result, locality is improved and the number of independent metadata accesses required to read VL data is reduced.

The descriptor format and datatype semantics remain unchanged, and VL handling continues to operate through the existing callback mechanisms described in Section 3.5.2. Nested VL datatypes, memory management behavior, and conversion logic are unaffected, as the only change is the location and organization of payload storage.

However, this approach introduces new responsibilities for managing heap space within each chunk. Fragmentation and compaction may need to be handled at the chunk level, and updates to VL data may require resizing or rebuilding the chunk-local heap. These tradeoffs are similar to those already present in the global heap but are now scoped per chunk.

Overall, this proposal replaces file-level global heap collections with chunk-local heap instances, improving locality and simplifying access patterns while preserving the existing VL abstraction and implementation structure.

The diagram below highlights that, unlike the current implementation, both descriptor and payload data are retrieved through a single chunk access. The heap abstraction is preserved, but its storage and lifecycle are scoped locally to the chunk rather than managed as a separate metadata object.



Each dataset element stores a descriptor containing a reference to a heap object, like the current implementation. However, rather than referencing a global heap collection, the descriptor resolves to a heap structure maintained locally within the chunk. This heap maintains the same internal structure as the current global heap (see Section 4), including an object table and a contiguous payload buffer.

Accessing the payload involves resolving the descriptor within the chunk-local heap, using the stored reference to locate the appropriate entry and retrieve the associated payload bytes. The resulting VL payload is then returned to the user.

Compared to the current model, this approach eliminates the need for metadata cache lookups during VL payload access, as both descriptors and payloads are retrieved as part of the chunk data. The heap abstraction is preserved, but its storage and lifecycle are scoped to the chunk rather than managed as a separate metadata object. As a result, it maintains compatibility with the existing heap-based design while improving locality by co-locating descriptors and payload storage within the same chunk.

5.2.1 Mapping of the Global Heap Components to Chunk-Local Design

The chunk-local heap model preserves the existing heap-based abstraction and adapts the current global heap implementation (H5HG) to operate within a chunk-local storage context rather than as a file-level metadata structure. More specifically, the likely reuse target is not the entire current H5HG subsystem unchanged, but the parts that manage an in-memory heap buffer, object table, allocation from free space, and local compaction. The metadata-cache-specific surrounding behavior, address-based protection, H5AC_GHEAP lifecycle, and independent heap serialization as a metadata object, would need to be replaced by chunk-relative handling. As described in Section 4, the global heap already provides a self-contained mechanism for managing variable-sized data within a contiguous buffer using an object table, linear allocation from a free-space region, and compaction through object relocation. This approach reuses those same mechanisms but relocates them into the chunk.

Under this design, each chunk contains an embedded heap region that functions similarly to a global heap collection. In memory, this region can be represented using structures analogous to `H5HG_heap_t`, including a buffer for payload data and an object table that tracks payload locations and free space. Allocation of VL payloads proceeds using logic equivalent to `H5HG__alloc`, where space is carved from a contiguous free-space region, and deletion follows the same compaction model as `H5HG_remove`, shifting objects and consolidating free space. As a result, the existing behavior of maintaining a densely packed heap with a single free region can be preserved without introducing a new allocation strategy.

The primary change lies in how the heap is stored and accessed. The heap becomes part of the chunk's data representation and is constructed and accessed as part of chunk I/O operations. During writes, the gather phase of the chunk cache pipeline builds an in-memory representation of the chunk that includes both descriptors and the heap region, inserting payloads into the heap and updating descriptor references accordingly. The serialize phase then encodes this combined structure into the chunk's on-disk format. During reads, the deserialize phase reconstructs the heap from the chunk data, and descriptor references are resolved directly within this local heap without requiring global heap lookups.

Operationally, this means the chunk-local heap would become part of the in-memory chunk representation. During gather, VL payloads associated with that chunk are placed into the chunk-local heap; during serialization, that heap would be compacted or encoded into the chunk's on-disk bytes; during deserialization, the chunk-local heap would be rebuilt in memory; and during scatter, the requested VL payloads would be copied back out into user-visible representations. Unlike the flattened model, descriptor references in the chunk-local heap approach remain stable across compaction operations, as object indices do not change when payloads are relocated within the heap. This reduces the need to update descriptors during layout changes and simplifies update handling.

Because the heap is embedded within the chunk, its lifecycle is tied to chunk access rather than metadata cache operations. This eliminates the need for metadata cache protections, file-address-based lookup, and separate heap serialization logic. Instead, heap state is naturally persisted and reconstructed as part of chunk encode/decode operations. This also ensures that all data required to

interpret VL values is available within a single chunk access, improving locality and reducing metadata overhead.

Fragmentation behavior differs from the global heap model. In the chunk-local model, fragmentation is isolated within individual chunks rather than shared across the file. This simplifies management and improves locality by keeping related data together.

Overall, this approach reuses the internal structure and algorithms of the global heap while replacing its metadata-cache-based storage model with a chunk-local representation integrated into the chunk cache pipeline. This allows the system to retain proven allocation and compaction behavior while improving locality and reducing reliance on global metadata structures.

Finally, the chunk-local heap model changes the scope of failure and corruption. Because heap data is localized to a chunk, any corruption in heap structure affects only that chunk rather than a shared global structure. This improves fault isolation while requiring consistency between descriptors and heap contents within each chunk.

One limitation of this implementation is compression efficiency. A single `H5_SECTION_VL` heap image may contain metadata, object-table information, local identifiers, nested descriptors, and payload bytes in the same section. This organization is good for implementation reuse, but it is not necessarily the best compression layout for every VL datatype. Compression should still be useful for common cases such as VL strings and simple non-nested VL data because the payload bytes are localized within the chunk. Nested VL and mixed compound/VL payloads may benefit from a future layout that separates heap metadata from payload bytes.

A future format could place the heap object table or local index metadata in its own section, or use a different versioned body inside `H5_SECTION_VL`, so that metadata and payload data can be filtered or compressed more effectively. That approach may improve compression but would also introduce additional section layout and indexing complexity. The first implementation keeps the object table and payload together in one chunk-local heap image to minimize risk and maximize H5HG reuse.

5.2.2 Integration with Structured Chunk Pipeline

In the chunk-local heap model, variable-length payload handling is no longer performed through the global heap and metadata cache, but instead is integrated directly into the structured chunk processing pipeline provided by the Shared Chunk Cache (SCC).

Under the current implementation, VL payload storage is triggered during datatype conversion (`H5T__conv_vlen`), which delegates to the VOL blob layer and ultimately to the global heap (H5HG) via `H5VL_blob_put` and `H5VL_blob_get`. This places payload allocation and retrieval outside of the dataset I/O pipeline and introduces additional metadata-driven operations.

In the chunk-local model, this responsibility shifts into the structured chunk pipeline. Rather than allocating payloads during datatype conversion, VL data is staged and incorporated during chunk processing through the structured chunk layout callbacks.

In the current structured chunk implementation, these stages correspond to layout-specific callbacks such as `H5D__struct_chunk_gather_mem`, `H5D__struct_chunk_encode`, `H5D__struct_chunk_decode`, and `H5D__struct_chunk_scatter_mem`, which collectively define how data is moved between application memory and chunk storage.

This results in the following redistribution of responsibilities:

- Gather Phase (`gather_mem` callback):

During gather, VL elements encountered in the input buffer are not written to an external heap. Instead, their payloads are inserted into a chunk-local heap structure being built in memory for the current chunk. The descriptor written into the chunk's fixed-length region contains a reference (e.g. object index) into this local heap. This replaces the current behavior where descriptors reference global heap objects.

- Serialize Phase (`encode` callback):

The chunk-local heap is serialized as part of the chunk image. This includes encoding the heap buffer and associated object metadata into a contiguous region that is stored alongside the dataset's fixed-length data. No metadata cache or external heap objects are involved.

- Deserialize Phase (`decode` callback):

When a chunk is loaded, the chunk-local heap is reconstructed in memory from the serialized chunk image. This restores the heap buffer and object table so that descriptor references can be resolved without external lookup.

- Scatter Phase (`scatter_mem` callback):

During scatter, VL descriptors are resolved against the in-memory chunk-local heap. Payload bytes are retrieved directly from the heap buffer and used to reconstruct application-visible VL values via the existing datatype conversion and allocation mechanisms.

This integration fundamentally changes where VL storage logic resides. Rather than being driven by datatype conversion callbacks and external heap operations, payload management becomes part of chunk construction and interpretation.

Importantly, this does not eliminate `H5T__conv_vlen` or the VL callback mechanism. Instead, it changes what those callbacks do. Operations that previously invoked `H5VL_blob_put` and `H5VL_blob_get` to access global heap storage must instead interact with chunk-local heap state provided during chunk processing.

This design aligns VL handling with chunk-level I/O, eliminates metadata-cache dependency for payload access, and ensures that all data required to reconstruct VL elements is available within a single chunk operation.

Direct structured-chunk I/O is a related future concern. Ordinary H5Dwrite/H5Dread should continue to let SCC/H5D construct and interpret sections internally. However, HDF5 also supports direct chunk access APIs, and future structured-chunk VL support may need helper functionality that packs user-provided section buffers into a valid structured-chunk image, including H5_SECTION_VL. This is not required for the first I/O path, but the section format should remain explicit enough to support such tooling later.

5.2.3 Versioning and Upgrade Path

The chunk-local heap representation should include versioned layout information in the serialized chunk image. This provides an upgrade path for future changes to the chunk-local VL format, including changes to heap organization, descriptor encoding, compaction policy, or section layout.

This is important because the initial implementation may favor simplicity such as compacting or rebuilding heap contents during chunk serialization. Future implementations may optimize this behavior by reducing unnecessary compaction, changing how free space is tracked, or modifying how heap metadata is stored. A versioned representation allows these changes to be introduced without breaking compatibility with earlier files.

The initial version should therefore define a clear on-disk structure for the fixed-length descriptor region, the chunk-local heap region, and any heap metadata required to reconstruct the in-memory heap. Later versions can refine these details while preserving the ability to recognize and read older chunk-local VL encodings.

5.2.4 Fractal Heap Storage Considerations

HDF5 also contains the H5HF fractal heap implementation. This is separate from the older H5HG global heap mechanism used by the current native VL storage path. The fractal heap code and private interfaces indicate that fractal heaps have their own format and policy state, including heap ID sizing, managed-object doubling-table parameters, direct-block checksum policy, tiny/managed/huge object handling, and filter pipeline settings.

This RFC does not select the fractal heap format for the first structured-chunk VL implementation. The v1 design is based on adapting the existing H5HG-style VL/global-heap path into a chunk-local H5_SECTION_VL body, because that path is closest to the current VL storage implementation and is expected to be the lowest-risk reuse target.

Currently, this RFC has not identified a complete documented mapping from the fractal heap file format into a chunk-local H5_SECTION_VL section body. In particular, the design would need to show how fractal heap headers, heap IDs, direct and indirect block organization, free-space state, checksums, filters, and related heap metadata would be serialized as part of the structured chunk image rather than as independent metadata objects.

Therefore, fractal heap support is acknowledged as related prior HDF5 heap infrastructure, but it is not part of the selected v1 design. The H5_SECTION_VL encoding should remain versioned so that a

later implementation can adopt a different heap representation if the required file-format and implementation analysis shows a clear advantage.

5.3 Discussion and Design Tradeoffs

The two approaches considered for reconstructing variable-length storage, reusing the global heap as chunk-local heaps and adopting a flattened chunk-local layout, represent fundamentally different strategies for addressing the indirection and metadata-cache dependency introduced by the current global heap design. Both approaches eliminate the need to resolve VL payloads through separate metadata structures, but they differ significantly in how much of the existing system they reuse and where complexity is introduced.

The chunk-local heap approach retains the core structure of the existing global heap implementation while relocating it into the context of the chunk. As shown in Section 4, the global heap already provides a well-defined mechanism for managing VL data, including allocation from a free-space region, compaction on deletion, and dynamic growth. The mapping in Section 5.2.1 demonstrates that these mechanisms can largely be reused, with the primary changes being the removal of metadata-cache-based lifecycle management and the replacement of global identifiers with chunk-local references. As a result, this approach preserves existing VL handling behavior while improving locality by co-locating descriptors and payload storage within the same chunk.

In contrast, the flattened chunk-local layout eliminates the heap abstraction entirely and replaces it with a direct offset-based encoding within the chunk. Under this model, each descriptor directly identifies its associated payload within the chunk's VL region. This produces a more self-contained representation in which all data associated with a chunk is stored together and resolved without heap lookup. From a conceptual standpoint, this model is simpler and more localized.

However, this simplification shifts responsibility for allocation, compaction, and space management into the chunk layer. Because payloads are referenced by offsets, any relocation of data requires updating descriptors, creating a tight coupling between layout and descriptor correctness. While a rebuild-on-write strategy can mitigate this complexity for an initial implementation, it still represents functionality that must be newly developed rather than reused.

A key distinction between the two approaches is how descriptor references behave under layout changes. In the chunk-local heap model, descriptor references remain stable across compaction operations, as object indices do not change when payloads are relocated within the heap. In contrast, the flattened model encodes direct offsets, requiring descriptor updates whenever payload layout changes. This difference has significant implications for update complexity and implementation effort. In particular, stable references reduce the need to propagate updates to descriptors during layout changes.

From an implementation perspective, the chunk-local heap approach provides a strong advantage in terms of reuse and implementation risk. The existing H5HG logic already implements the necessary behaviors for managing VL data, and adapting this logic to operate within a chunk-local context requires relatively targeted changes. In particular, the core allocation, compaction, and object-tracking

mechanisms can be preserved, reducing the amount of new code required and the likelihood of introducing subtle, unforeseen bugs.

An additional advantage of the chunk-local heap approach is that it builds directly on existing and already tested infrastructure. Because the core allocation, compaction, and object management logic is reused from the global heap implementation, much of the behavior has already been validated with existing testing facilities. This reduces the amount of new functionality that must be tested via brand new testing code. In contrast, the flattened model requires developing new layout and space management mechanisms, which would need to be tested and validated from scratch.

The flattened approach, while conceptually simpler, requires the introduction of new mechanisms for managing payload layout within the chunk. Although it removes heap indirection and improves locality, it does not eliminate the need for conversion between serialized and in-memory VL representations. As a result, the primary benefits of the flattened model are structural rather than representing a fundamental reduction in the cost of VL data reconstruction.

In terms of performance, both approaches eliminate metadata-cache-driven heap lookups during payload resolution, simplifying the access path. Differences between the approaches are therefore primarily structural and related to implementation complexity rather than fundamental changes in processing behavior.

Overall, the chunk-local heap approach provides a lower-risk path that reuses existing mechanisms and requires fewer structural changes to the system. The flattened approach offers a cleaner and more direct representation but introduces additional complexity and requires building new infrastructure for allocation and compaction. While both approaches are viable, the chunk-local heap model aligns more closely with the goal of minimizing change while improving locality, whereas the flattened model represents a more substantial redesign with potential long-term benefits.

6 Testing

Testing for the proposed chunk-local heap approach builds on existing HDF5 test infrastructure while introducing additional validation to ensure correct integration of variable-length datatypes with structured chunk storage. The current test suites already provide strong coverage of the individual components involved in this design, including heap management, VL datatype semantics, and structured chunk behavior. However, they do not independently validate the interaction between these components under the new storage model.

The overall testing strategy therefore consists of two parts: reusing existing tests to validate preserved behavior and introducing new tests to validate the interaction between VL datatypes and the structured chunk pipeline.

6.1 Existing Test Coverage

6.1.1 Global Heap (H5HG) Tests

The global heap (H5HG) test suite validates the core behaviors required for heap-based storage. These tests exercise allocation, insertion, removal, compaction, and reuse of freed space under a variety of

patterns, including monotonically increasing and decreasing object sizes, partial removal scenarios, and repeated insertion and deletion leading to index reuse and wraparound. They also verify correct reconstruction of heap state across file close and reopen operations.

Because the chunk-local heap approach reuses these same allocation and compaction mechanisms, but scopes them to individual chunks rather than file-level collections, these tests provide direct validation of the underlying heap behavior.

However, these tests operate in isolation and do not involve dataset I/O or datatype conversion. As a result, they do not validate how VL payloads are produced or consumed during actual dataset operations.

6.1.2 VL Datatype Tests

The VL datatype test suite (e.g. `tvltypes.c`) provides comprehensive validation of VL behavior at the API level. These tests cover VL sequences and strings, including handling of nil values, nested VL datatypes, and compound datatypes containing VL members. They also exercise rewriting datasets with larger and smaller payloads, validate fill value behavior across dataset layouts, and test memory management through custom allocation and reclamation routines such as `H5Treclaim` and `H5Dvlen_get_buf_size`.

These tests define the expected behavior of VL datatypes independent of the underlying storage mechanism and must continue to pass unchanged under the new implementation.

In their current form, however, these tests primarily exercise the global heap-backed VL path. They do not force execution through the structured chunk pipeline or validate chunk-local payload storage. To fully validate the new design, these tests must be executed or adapted to use structured chunk layouts so that identical VL behavior is verified under the new storage model.

6.1.3 Structured Chunk Tests

The structured chunk (sparse storage) test suite validates the infrastructure used for chunk-local storage. These tests exercise APIs such as `H5Dget_defined`, `H5Derase`, direct structured chunk I/O (`H5Dwrite_struct_chunk` and `H5Dread_struct_chunk`), chunk metadata queries, and structured chunk iteration. They also test interactions between structured chunk layouts and dense chunk APIs, as well as filtering behavior applied to structured chunk data.

These tests establish that the structured chunk pipeline and its associated callbacks operate correctly for fixed-size data.

However, this test suite is currently incomplete and contains multiple areas where validation is still marked as to be determined. In addition, it does not include VL datatypes and therefore does not validate the behavior of chunk-local heap storage. As a result, it provides a foundation for testing the execution path but must be extended to include VL-specific scenarios.

6.2 New Test Requirements

While existing tests validate individual components, they do not cover the combined behavior introduced by the chunk-local heap model. The primary gap is the lack of tests that exercise VL datatype semantics within the structured chunk execution model.

New tests must therefore validate the interaction between VL datatypes, chunk-local heap storage, and structured chunk processing.

End-to-end tests are required to verify correct behavior when VL datasets are stored using structured chunks. These tests must confirm that payloads are correctly inserted into the chunk-local heap during write operations, that descriptors correctly reference heap entries, and that payloads are properly reconstructed during read operations.

Attention must be given to update scenarios. Tests must validate overwriting VL elements with both larger and smaller payloads, ensuring correct heap allocation, reuse of freed space, and descriptor stability. Partial updates within a chunk must also be exercised to ensure that modifications to a subset of elements do not corrupt existing payloads or heap state.

Deletion behavior must be validated to confirm that removing VL elements correctly frees associated heap storage and that chunks are properly cleaned up when all elements are removed. Similarly, tests must ensure that structured chunk operations such as `H5Dget_defined` and `H5Derase` behave correctly in the presence of VL data.

Fill value handling requires additional validation. Tests must confirm that VL fill values are correctly represented in structured chunk datasets, including both null and non-null cases, and that they behave correctly when overwritten.

Persistence testing must verify that chunk-local heap state is correctly reconstructed after file close and reopen operations. This ensures that encode and decode logic correctly preserves heap structure and payload data within the chunk image.

More complex datatype scenarios must also be covered, including nested VL datatypes and compound types containing VL members. These tests ensure that recursive allocation and reconstruction behave correctly when payloads are stored within chunk-local heaps.

Finally, structured chunk tests must be extended to explicitly exercise VL behavior within the SCC pipeline. This includes validating correct operation of the `gather_mem`, `encode`, `decode`, and `scatter_mem` callbacks when processing VL data, as well as ensuring correct interaction with filtering, chunk iteration, and direct chunk access.

6.3 Summary

The proposed testing strategy leverages existing validation of heap mechanics, VL datatype semantics, and structured chunk infrastructure, while introducing targeted tests to ensure correct interactions between these components. Existing test suites provide strong coverage of individual behaviors, but new tests are required to validate their integration under the chunk-local heap model. This combined approach ensures that the new implementation preserves existing VL behavior while correctly supporting chunk-local storage and structured chunk processing.

7 Recommendation

Based on the analysis presented in Sections 5.1-5.3, the chunk-local heap model is the recommended approach for improving variable-length datatype storage in HDF5.

Both the flattened chunk-local layout and the chunk-local heap model eliminate the need for global heap lookups and improve locality by co-locating VL payloads with dataset data. However, the two approaches differ significantly in how they integrate with the existing system, the amount of new infrastructure required, and the level of risk introduced.

The flattened model offers a simpler conceptual representation by encoding payloads directly within the chunk using offsets. However, this approach requires introducing new mechanisms for payload compaction, allocation, and space management within the chunk. It also introduces tighter coupling between payload placement and descriptor correctness, requiring descriptor updates whenever layout changes occur. As a result, it represents a broader redesign of VL storage semantics and requires substantial new implementation effort, along with extensive validation of new correctness guarantees.

In contrast, the chunk-local heap model preserves the existing heap-based abstraction and maintains the current descriptor semantics. The primary change is the scope of heap storage, moving from file-level global heaps to chunk-local heaps. Allocation, compaction, and object tracking logic can be adapted from the existing H5HG implementation with relatively limited changes, while metadata-cache-based behavior is replaced with chunk-local lifecycle management.

This approach also aligns naturally with the structured chunk pipeline introduced by the Shared Chunk Cache (SCC). VL payload handling can be integrated into existing structured chunk callbacks, allowing payload construction, serialization, and reconstruction to occur as part of chunk processing. This avoids introducing new I/O pathways and ensures that VL storage integrates cleanly with the existing chunk-based execution model.

An additional advantage of the chunk-local heap approach is that it builds on existing, well-tested infrastructure. Because much of the core behavior is reused, the amount of new code required is reduced, along with the associated testing and validation effort. Descriptor references remain stable across compaction operations, simplifying update behavior and reducing the likelihood of subtle correctness issues.

Since both approaches achieve the central goal of co-locating VL payloads with chunk data, the preferred approach is the one that does so with the fewest semantic changes and the greatest reuse of existing, tested mechanisms. On that basis, the chunk-local heap model is the stronger initial implementation choice.

For these reasons, development should proceed with the chunk-local heap model, focusing on adapting H5HG heap management logic to operate within a chunk-local context and integrating it into the structured chunk processing pipeline.

8 Chunk-Local VL Heap Implementation Mapping

This section describes the implementation map for storing variable-length payloads inside structured chunks. The goal is to preserve the existing HDF5 VL API, H5T conversion semantics, memory allocation/reclaim behavior, and nested VL behavior while replacing the storage backend used by structured-chunk VL datasets.

The implementation keeps VL semantics in the datatype layer, reuses H5HG heap mechanics where practical, and integrates payload storage into the structured-chunk section pipeline. The legacy global-heap path remains available for existing files and layouts that do not use structured-chunk VL storage.

8.1 Implementation Overview

The implementation changes the storage endpoint for structured-chunk VL datasets without changing the external VL API or the memory-side representation. The current native path stores a fixed-size descriptor in dataset storage and uses that descriptor to reach a VOL blob/global-heap object. The structured-chunk path keeps the fixed-size descriptor but interprets its reference field as a chunk-local heap reference while SCC is processing a structured chunk.

Current native path:

```
Descriptor in dataset storage
    ->VOL blob ID
        ->H5VL_blob_get/put/specific
            ->H5HG global heap collection in metadata cache
```

Chunk-local structured-chunk path:

```
Descriptor in H5_SECTION_FIXED
    ->local heap index encoded in descriptor reference field
        ->H5HG local heap helpers
            -> H5_SECTION_VL inside the same structured chunk
```

For top-level VL elements, H5_SECTION_FIXED stores the root fixed-size descriptor. If the payload itself contains nested VL values, nested descriptors may live inside heap-stored data in H5_SECTION_VL. For that reason, SCC must not parse nested VL payloads directly – H5T’s recursive VL traversal must remain responsible for nested VL semantics.

H5_SECTION_FIXED is used for root VL descriptors because the descriptors are still fixed-size file records. For a VL datatype, the variable-sized payload bytes are not stored inline; the inline record is a fixed-size descriptor containing the sequence length and a reference field. In the structured-chunk VL path, that reference field is interpreted as a chunk-local heap reference. The payload bytes themselves are stored in H5_SECTION_VL. A separate descriptor section is unnecessary for the initial implementation because it would duplicate the role of H5_SECTION_FIXED without introducing a distinct storage category. This is also useful for compound datatypes that contain both ordinary fixed-size members and VL members: the fixed section remains the place for the fixed-size file representation, while H5_SECTION_VL stores out-of-line payloads.

The initial implementation preserves the existing disk VL descriptor size, 4 + blob_id_size. In the structured-chunk VL path, the bytes after the first four are interpreted as a chunk-local heap reference

rather than a VOL blob/global-heap identifier. This avoids a wider ripple through descriptor-size calculations, selection I/O buffers, and conversion paths.

8.2 Conceptual Overview and Responsibility Split

The baseline constraints for the VL redesign are:

- Fixed-size disk descriptors must remain compatible with ordinary HDF5 dataset I/O
- Nested VL semantics must be preserved
- Memory allocation and reclaim must continue to use the existing H5T machinery
- The new path must coexist with the legacy global-heap path
- Callbacks must operate on caller-owned decoded chunk state rather than owning chunk-object lifetime

The implementation uses the following responsibility split:

Component	Responsibility
H5T	Owns VL semantics: descriptor interpretation, recursive traversal, nested VL behavior, base-type conversion, memory allocation policy, and reclaim behavior.
H5T Chunk Path	Adds a disk-like VL backend for structured chunks. This redirects storage-end operations from VOL blobs/global heaps to the current chunk-local heap.
H5HG	Owns reusable heap mechanics: object table, heap buffer, allocation, removal, extension, compaction, and heap image encode/decode support.
SCC/H5D	Owns the decoded chunk lifecycle, section splitting and concatenation, gather/scatter, encode/decode, erase/fill, filtering/checksum integration, and cache admission sizing.
Decoded Chunk	Contains optional chunk-local heap state. The caller/cache owns the chunk object; callbacks operate on it.

The implementation should use H5_SECTION_* enum values directly as indices into offset[], unfilt_size[], and filt_mask[], because the current structured-chunk index handling already treats those enum values as direct array indices. At the same time, the code should remain careful about the

distinction between H5O_MAX_STC_NSECTS and H5_SECTION_NUM, and it should replace raw 0 and 1 indexing with named section constraints everywhere it touches VL support.

8.3 Data Structures

8.3.1 Decoded Chunk State

The decoded structured-chunk state needs one optional heap pointer and optional VL size counters. The heap pointer is optional because a chunk may contain selection/fixed data while having no non-null VL payloads.

```

/*****
 *
 * struct H5D_chunk_cache_mem_t
 *
 * H5D_chunk_cache_mem_t is the decoded in-memory representation of a structured
 * chunk while that chunk is resident in the shared chunk cache or being processed
 * by the structured-chunk layout code.
 *
 * The selection fields describe which logical elements in the chunk are defined.
 * The fixed-data fields store the fixed-size records for those defined elements.
 * For VL datasets, the fixed-size records contain root VL descriptors. The
 * variable-length payload bytes referenced by those descriptors are stored in the
 * optional chunk-local heap and serialized into H5_SECTION_VL during encode.
 *
 * This structure owns its buffers. If vl_heap is non-NULL, this structure also
 * owns the chunk-local heap. H5T VL callbacks may borrow access to vl_heap during
 * gather, scatter, fill, overwrite, or erase, but they must not own the decoded
 * chunk object and must not free the heap object directly.
 *
 * data_buf:      buffer containing fixed-size records for the defined elements
 *
 *                in the chunk. For VL datasets, this buffer stores root VL
 *
 *                descriptors, not the out-of-line VL payload bytes.

```

*
* sel_buf: temporary buffer containing the encoded selection section while
* the section is being decoded or encoded. The durable decoded
* selection state is represented by sel_space.
*
* sel_space: dataspace selection describing which logical elements in the
* chunk are currently defined.
*
* sel_nbytes: logical number of bytes in the selection section after filters
* and checksum metadata have been removed.
*
* sel_alloc_size: number of bytes allocated for sel_buf.
*
* data_nbytes: logical number of bytes used in data_buf for fixed-size records.
*
* data_alloc_size:
* number of bytes allocated for data_buf.
*
* vl_heap: optional decoded chunk-local heap. This field is NULL when the
* chunk has no non-null VL payloads. When non-NULL, the heap is
* owned by this decoded chunk object and is serialized into
* H5_SECTION_VL during encode.
*
* vl_nbytes: logical number of bytes in the VL section payload after filters
* and checksum metadata have been removed. This is zero when
* vl_heap is NULL or empty.
*
* vl_alloc_size: number of bytes allocated for temporary VL-section storage, if

```

*          such a buffer is tracked by the implementation. This is zero
*
*          when no temporary VL-section buffer is allocated.
*
*****/
typedef struct H5D_chunk_cache_mem_t {
    /* existing fields */

    void  *data_buf;

    void  *sel_buf;

    H5S_t *sel_space;

    /* size tracking */

    size_t sel_nbytes;

    size_t sel_alloc_size;

    size_t data_nbytes;

    size_t data_alloc_size;

    /* new for VL structured chunks */

    H5HG_heap_t *vl_heap;

    size_t      vl_nbytes;

    size_t      vl_alloc_size;
} H5D_chunk_cache_mem_t;

```

8.3.2 Operation-Local VL Chunk Context

The operation-local VL chunk context carries the chunk-specific state that the chunk-local VL callbacks need while a single structured-chunk operation is running. This context is not part of the datatype object and is not part of the metadata cache. It is temporary state installed by the structured-chunk path before invoking H5T conversion and removed after that conversion finishes.

The context carries a pointer to the owning heap pointer, not just a borrowed heap value. This is required because the first non-null VL write may lazily create the chunk-local heap and store the new heap pointer back into the decoded chunk object.

```

/*****
*

```

```

* struct H5T_vlen_chunk_ctx_t
*
* H5T_vlen_chunk_ctx_t is temporary operation-local state used by the
* chunk-local VL callback table while H5T conversion is processing one
* structured chunk.
*
* The structure gives H5T chunk-local VL callbacks access to the decoded
* chunk-local heap for the chunk currently being gathered, scattered, filled,
* overwritten, or erased. It is installed before the conversion operation begins
* and removed on all success and failure exit paths after conversion completes.
*
* This structure does not own the file pointer, the heap pointer, or the
* datatype. The decoded chunk owns the heap. The callbacks borrow the heap
* through this context and may update the decoded chunk's heap pointer when
* lazy heap creation is required.
*
* f:          borrowed file pointer used by local H5HG helper routines for
*              encoding sizes, alignment rules, and other file-format-dependent
*              heap operations. The context does not own or close this file.
*
* heap:       pointer to the decoded chunk's vl_heap field. This is a pointer
*              to the owning heap pointer rather than a copy of the heap pointer
*              so that H5T write callbacks can lazily create the heap and store
*              it back into the decoded chunk object.
*
* ref_nbytes: number of bytes in the descriptor reference field after the
*              four-byte sequence-length field. For the first implementation,
*              this is the current blob-ID-sized field. The chunk-local backend
*              uses this region to encode and validate a local heap object index.
*
*****/

typedef struct H5T_vlen_chunk_ctx_t {
    H5F_t      *f;

    H5HG_heap_t **heap;

    size_t      ref_nbytes;

```

```
} H5T_vlen_chunk_ctx_t;
```

8.3.3 Ownership Rules

vl_heap is owned by the decoded chunk object, not by H5T and not by H5HG. H5T borrows access to it through operation-local context. H5HG provides helper functions that operate on it. SCC new-chunk and decode initialize it, gather/scatter/erase/fill mutate it, encode serializes it, and eviction frees it.

H5T callbacks must not directly free or null out the owning chunk field. If the heap becomes empty SCC may free it and reset the field to NULL after the operation.

8.3.4 Decoded Local Heap RAM Format

The decoded chunk-local VL heap should use an H5HG-style in-memory representation. While a structured chunk is resident in memory, the decoded chunk owns an H5HG_heap_t through chk->vl_heap. That heap object is the RAM representation of H5_SECTION_VL.

This means the decoded heap is not represented as a flat array of payload bytes. It is represented as a heap object with a contiguous heap buffer and object table, following the current H5HG model:

```
H5HG_heap_t
    heap->chunk
        contiguous heap image buffer
    heap->obj[]
        object table entries
        each allocated object entry points into heap->chunk
    heap->obj[0]
        reserved free-space object
```

The current H5HG implementation uses heap->chunk as the contiguous memory image of the heap collection and uses heap->obj[] to locate individual heap objects inside that image. The chunk-local VL path should preserve that RAM model where practical. Local heap object indices stored in VL descriptors refer to entries in heap->obj[].

For example:

```
root VL descriptor in H5_SECTION_FIXED
    local heap index = 5
```

```
chk->vl_heap->obj[5]
    begin pointer into chk->vl_heap->chunk
    size of heap object payload
```

```
heap object payload
    VL element bytes
    possibly including nested VL descriptors
```

The decoded chunk owns `chk->vl_heap`. H5T callbacks may read, insert, or remove objects through local H5HG helpers, but they should treat the heap as chunk-owned state. H5HG helper functions operate on this RAM representation; they do not make the heap a standalone metadata object and do not give it independent file-space ownership.

When the chunk is encoded, `H5HG__encode_local()` serializes this RAM representation into logical `H5_SECTION_VL` payload. When the chunk is decoded, `H5HG__decode_local()` reconstructs the `H5HG_heap_t`, `heap->chunk`, and `heap->obj[]` state from the logical `H5_SECTION_VL` payload.

8.4 Structured-Chunk Section Layout and Descriptor Format

The structured-chunk VL path uses the existing structured-chunk section model but extends the encoded chunk image to include an optional VL payload section. The exact section list depends on the structured-chunk storage mode. Sparse fixed-size chunks use selection and fixed sections. Sparse VL chunks use selection, fixed, and VL sections. Dense VL chunks use fixed descriptors and a VL heap section, without a sparse selection section.

Section layout summary:

- Sparse fixed-size = `H5_SECTION_SELECTION` + `H5_SECTION_FIXED`
- Sparse VL = `H5_SECTION_SELECTION` + `H5_SECTION_FIXED` + `H5_SECTION_VL`
- Dense VL = `H5_SECTION_FIXED` + `H5_SECTION_VL`

`H5_SECTION_SELECTION`

Tracks which logical elements in the chunk are defined.

`H5_SECTION_FIXED`

Stores fixed-size records for the defined elements. For VL datatypes, this section stores root VL descriptors, not out-of-line payload bytes.

`H5_SECTION_VL`

Stores the serialized chunk-local heap. For nested VL datatypes, heap payloads may themselves contain nested descriptors or references, and those nested references resolve within the same chunk-local heap.

The fixed section does not become variable-sized because the datatype contains VL. It continues to hold fixed-size file records. The variable-sized payload bytes move to `H5_SECTION_VL`.

8.4.1 H5_SECTION_VL Image Format

The serialized H5_SECTION_VL image must be self-describing enough to support future format changes, but the initial implementation should keep additional wrapper metadata minimal. The first implementation should reuse as much of the existing H5HG serialized heap image as practical.

If the existing H5HG image already contains sufficient information to identify and decode the heap body, then H5_SECTION_VL does not need a separate local wrapper header. If additional identification or versioning is required, the section should contain a small version marker or wrapper before the heap body.

Conceptually, a non-empty H5_SECTION_VL contains:

[optional VL-section version/wrapper][serialized chunk-local H5HG-style heap image]

This optional wrapper is local to the H5_SECTION_VL payload. The dataset-level sectioned storage header remains the existing data layout message version 5 sectioned-storage header.

The purpose of this versioning metadata is to distinguish the initial chunk-local heap representation from later representations. The implementation should not bake in assumptions that would prevent a later flattened VL section, a different heap organization, or a different compaction policy from being added under a new version.

Checksum handling for H5_SECTION_VL follows the metadata-section policy described later in Section 8.7. The VL heap format should not duplicate checksum logic unless SCC cannot protect the section as metadata.

8.4.2 Structured-Chunk VL Descriptor Format

For the initial implementation, the structured-chunk VL descriptor should preserve the existing disk VL descriptor size. The first four bytes continue to store the sequence length. The remaining bytes continue to occupy the current blob-ID-sized reference field, but in the structured chunk VL path those bytes are interpreted as a local heap object index rather than as a VOL blob/global-heap identifier.

Structured-chunk VL descriptor:

[4-byte sequence length][local index encoded in blob-id-sized field]

The null encoding is:

Sequence length = 0

Local reference field = all zero bytes

The non-null encoding is:

Sequence length > 0

Local reference field = non-zero local heap object index

Mismatched encodings should be treated as corrupt data. Examples include:

Sequence length = 0 with non-zero local reference

Sequence length > 0 with all-zero local reference

Preserving the existing descriptor size avoids a wider ripple through descriptor-size calculations, selection I/O buffers, and conversion paths. A later format version may shrink the reference field or replace it with an explicitly tagged local descriptor if there is a strong reason to do so.

8.4.3 Non-Empty VL Heap Layout

When a chunk exists and contains a non-empty VL heap section, the logical section layout is:

offset[H5_SECTION_SELECTION] = 0

offset[H5_SECTION_FIXED] = sel_section_encoded_nbytes

offset[H5_SECTION_VL] = sel_section_encoded_nbytes + fixed_section_encoded_nbytes

H5_SECTION_SELECTION	H5_SECTION_FIXED	H5_SECTION_VL
Selection metadata	Fixed records / root VL descriptors	Optional wrapper + serialized chunk-local heap image

For sparse VL datasets, H5_SECTION_FIXED contains the root fixed-size records for defined elements. For VL elements, those records are root VL descriptors. H5_SECTION_VL contains the heap objects referenced by those descriptors. Dense VL follows the same fixed-descriptor plus VL-heapsplit but does not include a sparse selection section.

For nested VL datatypes, heap objects stored in H5_SECTION_VL may themselves contain nested VL descriptors or references. Those nested references also resolve within the same chunk-local heap. SCC should not parse those nested payloads directly – H5T’s recursive traversal remains responsible for nested VL semantics.

8.4.4 Zero-Sized VL Section

When the chunk exists but has no non-null VL payloads, H5_SECTION_VL may be zero-sized:

offset[H5_SECTION_VL] = total_encoded_chunk_nbytes

unfilt_size[H5_SECTION_VL] = 0

H5_SECTION_SELECTION	H5_SECTION_FIXED
----------------------	------------------

Selection metadata	Fixed records / root
	VL descriptors

This case can occur when the chunk contains defined fixed records or root VL descriptors but no non-null VL heap objects. In that case, the chunk remains present, but no heap image is serialized.

8.4.5 Empty Chunk Rule

A zero-sized H5_SECTION_VL is different from a completely empty chunk. If the chunk has no defined values, there should be no chunk entry at all. The chunk should not be materialized solely to preserve an empty VL heap section.

The rule is:

Chunk has defined values and non-null VL payloads:

store selection section, fixed section, and non-empty VL section.

Chunk has defined values but no non-null VL payloads:

store selection section and fixed section.

encode H5_SECTION_VL as zero bytes.

Chunk has no defined values:

omit the chunk entry entirely.

This separates section-level optionality from whole-chunk emptiness. H5_SECTION_VL may be absent or zero-sized inside an existing chunk, but an entirely empty chunk should not be stored just to represent an empty heap.

8.4.6 File-Format Header Alignment

The chunk-local VL heap path should use the existing sectioned-storage file-format machinery. The dataset-level layout information still comes from the sectioned-storage data layout message. For VL structured chunks, that layout information should record that the dataset uses variable-length sectioned storage, that the chunk has selection, fixed, and VL sections, and that the required sections are metadata-bearing.

Any chunk-local heap wrapper or version marker belongs inside the H5_SECTION_VL payload. It does not replace the normal sectioned-storage layout information.

For sparse VL structured chunks, the sectioned-storage header information should identify three logical sections:

- H5_SECTION_SELECTION
- H5_SECTION_FIXED

H5_SECTION_VL

It should also identify all three as metadata-bearing for VL datasets:

```
nsects = H5_SECTION_NUM
nsects_md = H5_SECTION_NUM
seq_sects_md[0] = H5_SECTION_SELECTION
seq_sects_md[1] = H5_SECTION_FIXED
seq_sects_md[2] = H5_SECTION_VL
```

This means the file-format-level header describes the presence of the VL section and the metadata/checksum policy for the sectioned chunk. It does not describe each local heap object. The detailed heap object layout is local to the H5_SECTION_VL payload and is handled by H5HG__encode_local() and H5HG__decode_local().

If the existing H5HG collection image is reused directly as the H5_SECTION_VL payload, then the H5_SECTION_VL payload begins with the reused H5HG-style heap image. If that image is not sufficiently self-describing for chunk-local use, H5_SECTION_VL may begin with a small chunk-local heap wrapper header followed by the H5HG-style heap body. That wrapper is part of the H5_SECTION_VL payload, not a replacement for the sectioned-storage layout header.

8.5 Theory of Operation

The following operation descriptions define the implementation map for the structured-chunk VL path. They are intentionally explicit so that heap creation, mutation, omission, deletion, and serialization are visible at each stage.

The RFC already describes the current H5T read/write paths, the background-buffer requirement for VL overwrite, and the legacy global-heap indirection. This section describes the replacement storage-end behavior for structured-chunk VL datasets only. It does not change the external VL API, memory-side representation, or recursive H5T conversion semantics.

At a high level, writes enter the structured-chunk path through gather, where SCC/H5D stages user data into a caller-owned decoded chunk object. For VL datasets, gather temporarily routes file-side VL storage operations through the chunk-local VL backend, gives that backend access to the current decoded chunk heap, and lets the existing conversion path create or mutate the optional chunk-local heap. Encode then serializes the selection section, fixed descriptor section, and optional VL heap section into the final structured-chunk image.

Reads perform the inverse. Decode splits the stored chunk image into sections, reverses filters, verifies metadata-bearing sections, and reconstructs the optional chunk-local heap from H5_SECTION_VL. Scatter then routes file-side VL reads through the chunk-local backend so that user-

visible `hvl_t` values or VL strings are reconstructed from the chunk-local heap rather than from the legacy global-heap path.

8.5.1 New Chunk

When a new decoded chunk object is created for a structured-chunk VL dataset, it should initialize the VL heap state to empty:

```
chk->vl_heap      = NULL;
chk->vl_nbytes     = 0;
chk->vl_alloc_size = 0;
```

The heap is not eagerly created. Lazy creation is preferred because a chunk may contain only null VL descriptors, only fixed records from compound members, or no non-null VL payloads at all. In those cases, there is no reason to materialize chunk-local heap infrastructure.

For a structured-chunk dataset whose file datatype contains VL, the decoded chunk should be prepared for the three-section model described in Section 8.4:

H5_SECTION_SELECTION

H5_SECTION_FIXED

H5_SECTION_VL

H5_SECTION_VL may encode as zero bytes, but the decoded representation should still be able to track optional heap state consistently.

8.5.2 Decode

Decode converts the serialized structured-chunk section image into caller-owned decoded chunk state. For a VL structured chunk, decode should:

1. Determine whether the dataset file datatype contains VL.
2. Compute section extents from `offset[]` and the total encoded chunk size.
3. For a VL dataset, split the chunk into selection-section bytes, fixed-section bytes, and optional VL-section bytes.
4. Reverse section-level filters into per-section unfiltered buffers sized according to SCC's `unfilt_size[]` semantics.
5. Verify checksums on all metadata-bearing sections present in the image using the SCC section checksum mechanism described in Appendix E of HDF5 File Format Specification Version 3.1 RFC (Reference #5). In the current sparse fixed-size path this corresponds to the checksum protection for the encoded selection section. The VL path should reuse or refactor that

handling for H5_SECTION_FIXED and H5_SECTION_VL when those sections are metadata-bearing.

6. Remove checksum bytes or checksum metadata from the logical section payloads according to SCC rules.
7. Decode selection state into the decoded chunk object.
8. Copy fixed-section bytes into the decoded chunk's fixed-data buffer.
9. If the VL section size is non-zero, pass its logical bytes to `H5HG__decode_local()` and store the resulting heap pointer in `chk->vl_heap`.
10. If the VL section size is zero, leave `chk->vl_heap = NULL`.
11. Record decoded logical sizes such as `chk->sel_nbytes`, `chk->data_nbytes`, and `chk->vl_nbytes`, excluding checksum bytes or checksum metadata.

This is the natural inverse of encode. It also matches the SCC ownership model: decode fills fields in a caller-owned chunk object instead of owning the object outright.

8.5.3 Scatter Memory Read

Scatter reconstructs user-visible memory from the decoded chunk object. For VL datasets, scatter should:

1. Use the caller-owned decoded chunk object already built by decode.
2. Make sure `chk->vl_heap` is present if any non-null VL descriptor is about to be read.
3. Treat a non-null descriptor with `chk->vl_heap == NULL` as corrupt data.
4. Use a temporary patched view or copy of the dataset file datatype for structured-chunk VL conversion. This temporary patched datatype state may be created for the operation or cached as non-shared structured-chunk state.
5. Patch the temporary datatype state so every file-side VL node uses `H5T_vlen_chunk_g` rather than `H5T_vlen_disk_g`. The dataset's real/shared file datatype must not be permanently mutated.
6. Push operation-local context containing the file pointer, a pointer to the current chunk heap field, and the local reference-field size.
7. Call the existing type-conversion/scatter path.
8. Pop the operation-local context.

9. Release the temporary patched datatype copy if it was created for this operation; if a temporary patched datatype is cached, keep it as non-shared structured-chunk state without chunk-specific heap state.

This preserves current H5T read semantics. H5T still allocates user-visible VL buffers through the appropriate allocation path, still performs required base-type conversion, and still reconstructs hvl_t or VL strings in the user buffer. The only storage-end change is that H5T__vlen_chunk_read() resolves payloads from the caller-owned chunk-local heap instead of performing a VOL blob/global-heap lookup.

The durable rule is that the current chunk heap is supplied through operation-local context each time scatter/gather runs. The patched datatype itself may be per-operation or cached privately, but chunk-specific state must not be stored in the datatype object.

8.5.4 Gather Memory Write

Gather stages user memory into the decoded chunk object. For VL datasets, gather should:

1. Ensure the decoded chunk object exists.
2. Preserve background bytes for destination fixed descriptors when the write is an overwrite. VL overwrite still requires the previous descriptor so the old out-of-line payload can be deleted.
3. Use a temporary patched view or copy of the dataset file datatype for structured-chunk VL conversion. This temporary patched datatype state may be created for the operation or cached as non-shared structured-chunk state.
4. Patch the temporary file datatype so every file-side VL node uses H5T__vlen_chunk_g.
5. Push operation-local context containing the file pointer, a pointer to the current chunk heap field, and the local reference-field size.
6. Call the existing gather/conversion path.
7. Pop the operation-local context.
8. Free the temporary datatype copy.
9. After conversion, if `chk->vl_heap` exists but `H5HG__is_empty_local(chk->vl_heap)` is true, free it and reset `chk->vl_heap = NULL`.

This preserves the existing VL overwrite rule from the current write path: the old destination descriptor must be available during conversion so the old out-of-line payload can be deleted before or during installation of the new payload. The new path keeps that behavior but performs deletion and insertion against the chunk-local heap.

8.5.5 Overwrite

Overwrite is not a separate algorithm. It is a write where a background descriptor exists. The steps are:

1. Obtain and preserve the old file-side fixed descriptor bytes for all overwritten elements.
2. During conversion, allow the H5T chunk-local VL path to inspect the background descriptor.
3. Delete old local heap objects before or during installation of the new payload.
4. Insert the new payload into the chunk-local heap.
5. Update the fixed descriptor with the new length and local heap reference.
6. After conversion, check whether the heap has become empty and delete it if so.

The implementation must verify that nested VL cleanup is still handled recursively. If the existing H5T background-aware conversion path only deletes a single root heap object, then a file-side recursive delete helper must be added so child VL descriptors stored inside heap payloads are also released. SCC should not manually parse nested VL payloads.

This is the same reason the ordinary VL write path requires a background buffer today: overwriting only the fixed-size bytes is incorrect because the old out-of-line payload must be explicitly removed.

8.5.6 Erase

Erase is distinct from overwrite because there is no replacement value. The correct selective-erase behavior for VL data is:

1. If erase removes every defined value in the chunk, drop the chunk entry and free the entire decoded chunk object, including `vl_heap`. No per-element recursive heap cleanup is needed because the whole chunk-local heap is discarded with the chunk.
2. Otherwise, locate each erased fixed record using the existing structured-chunk mapping from logical element to fixed record.
3. For each erased record, invoke a recursive file-side VL delete helper against the erased element's file representation.
4. Remove the fixed record and update the selection section using existing SCC erase logic.
5. If the heap becomes empty after erase, free it and set `chk->vl_heap = NULL`.

Selective erase is the operation where a plain “remove the root object index” algorithm is most likely to be insufficient. For nested VL, child descriptors may exist inside heap-stored payload bytes in `H5_SECTION_VL`. A file-side recursive delete helper is therefore part of the implementation surface in H5T, mirroring the role that recursive reclaim logic plays for memory-side cleanup.

8.5.7 Fill

Fill has two cases.

For null VL fill, the chunk should write a zero-length descriptor with an all-zero local reference and should not create a heap for the filled value.

For non-null VL fill, the fill value should be routed through the same temporary patched-file-type path used by gather. This allows nested VL fill values to be handled by H5T recursion and the chunk-local write callbacks rather than by SCC-specific ad hoc code.

If fill overwrites existing non-null VL values, the old file-side descriptors must be available so the old heap objects can be deleted before the new fill value is installed. After fill, if the heap exists but is empty, SCC should free it and set `chk->vl_heap = NULL`.

This keeps fill behavior consistent with ordinary write behavior and avoids duplicating nested-VL storage logic in SCC.

8.5.8 Encode and Flush

Encode converts the caller-owned decoded chunk object back into the serialized structured-chunk section image. For VL structured chunks, encode should:

1. Encode the selection section payload.
2. Copy fixed descriptor bytes from the decoded chunk's fixed-data buffer.
3. If `chk->vl_heap` is non-NULL and non-empty, call `H5HG__encode_local()` to produce the logical `H5_SECTION_VL` payload.
4. If `chk->vl_heap` is NULL or empty, set the logical VL payload size to zero.
5. Protect every metadata-bearing section using the SCC checksum mechanism.
6. Apply section-level filters independently to each section if configured.
7. Concatenate filtered section images in `H5_SECTION_*` order.
8. Fill `offset[]`, `unfilt_size[]`, and `filt_mask[]` using `H5_SECTION_*` enum indices.
9. If the chunk has no defined values, do not store a chunk entry at all.

For the VL section payload, `H5HG__encode_local()` should first attempt to reuse an existing H5HG-style heap image directly. If the reused H5HG image is not self-describing enough for the structured-chunk VL section, the section may add a minimal local wrapper containing version/format information before the heap body. The exact wrapper format should remain local to `H5HG__encode_local()` / `H5HG__decode_local()` and should not affect H5T descriptor semantics.

Once the section image has been assembled, the final file write remains a normal chunk write or block write through the existing SCC/layout machinery. The special VL behavior ends when the chunk image has been built.

8.5.9 Delete Chunk

Whole-chunk deletion is simpler than selective erase. If a chunk is removed because all values are erased or because the chunk goes out of scope, SCC can discard the entire decoded chunk object and remove the chunk entry from the index.

For VL structured chunks, whole-chunk deletion should free the chunk-local heap wholesale. It does not need to recursively delete each individual VL payload object, because the heap is local to the chunk and no other chunk should reference its objects.

This rule is only valid for whole-chunk deletion. Selective erase still needs recursive file-side VL cleanup before removing individual fixed records.

8.5.10 Evict and Cleanup

Eviction frees layout-owned decoded chunk state. For VL structured chunks, that includes `vl_heap` if present. If temporary section buffers are allocated during decode or encode, they must also be freed.

The durable decoded representation should be the heap object, not a retained encoded-section byte buffer. This matches the SCC ownership model: the cache/caller owns the chunk object, and the layout-specific cleanup path frees layout-owned substructures inside that object.

Error paths should follow the same ownership rule. If decode, gather, scatter, fill, or encode fails after allocating temporary datatype copies, operation-local context, temporary section buffers, or a chunk-local heap, those resources must be released before returning failure.

8.5.11 Condense and Size Finalization

Condense and related size-finalization paths should include the optional VL section size in encoded-size and buffer-compaction calculations. If the heap is empty at this point, the final decoded state should be normalized to:

```
chk->vl_heap      = NULL;
chk->vl_nbytes     = 0;
chk->vl_alloc_size = 0;
```

If the whole chunk becomes empty after condense, the chunk entry should be removed rather than persisting a zero-information chunk. The guiding rule is that section sizes should always be known by the time encode/condense is running, at least in uncompressed form.

8.5.12 Encode-in-Place Policy

The policy for the initial implementation should explicitly disable encode-in-place for VL structured chunks. Heap compaction and the optional presence of a third section mean section sizes and offsets can change in ways that are not a safe fit for an in-place byte-shift optimization.

Any path that would otherwise use encode-in-place should fall back to the ordinary encode path for chunks whose file datatype contains VL. This is a deliberate implementation-safety policy, not a fundamental format limit.

8.5.13 Vector I/O Policy

The initial implementation should explicitly return “not possible” or use the existing fallback mechanism for vector I/O on VL structured chunks. Until the ordinary gather/scatter path is proven with context injection, checksums, optional heap omission, and nested VL semantics, the optimized vector path should not be enabled.

This keeps the first deliverable focused on one correct path.

8.5.14 SWMR Implications

The current VL storage model splits visibility across two storage domains: the fixed-size descriptor lives in dataset storage, while the payload lives in a separate global heap object. This makes it difficult to keep descriptor and payload visibility synchronized.

The chunk-local heap path improves this substantially for structured chunks because the descriptor section and heap section are encoded into the same chunk image and become visible together through the chunk path rather than through a chunk-plus-global-heap combination.

This does not create a stronger atomicity guarantee than the underlying structured-chunk write path already provides. However, it removes the current global-heap coordination problem from the structured-chunk VL path.

8.6 Detailed Code Mapping

The code-change surface is larger than the callback table alone. In addition to new H5T and H5HG helpers, the implementation must update structured-chunk section registration, checksum verification, overwrite background handling, cache-admission sizing, and fallback policies for optimized paths that are not supported in the first implementation.

8.6.1 Summary of Code Areas

Area	Existing object or function family	Required change
------	------------------------------------	-----------------

H5T VL backend	H5T_vlen_disk_g, disk VL callbacks, H5T__vlen_set_loc() path	Add H5T_vlen_chunk_g, local-reference encode/decode helpers, temporary datatype patch helper, and operation-local context API.
H5T recursive cleanup	Memory-side reclaim and nested VL conversion	Add or verify file-side recursive delete support for selective erase and nested VL overwrite cleanup.
H5HG	H5HG_insert, H5HG_read, H5HG_remove, H5HG_extend, H5HG_heap_t	Add local heap-pointer helpers that reuse allocation, compaction, growth, and object-table logic while skipping metadata-cache and file-address identity.
H5HG section image	Current metadata-cache heap serialization	Add H5HG__encode_local() and H5HG__decode_local() for H5_SECTION_VL; use direct H5HG image reuse if sufficient, otherwise add a minimal local version wrapper.
Decoded chunk object	H5D_chunk_cache_mem_t or equivalent decoded chunk representation	Add optional vl_heap, vl_nbytes, and vl_alloc_size fields.
Section metadata	offset[], unfilt_size[], filt_mask[], metadata-section registration	Add/enable H5_SECTION_VL; use enum values as section indices; ensure fixed and VL sections are protected as metadata-bearing sections for VL datasets.
Structured chunk lifecycle	H5D__struct_chunk_set_sizes, new_chunk, decode, encode, gather_mem, scatter_mem, erase_values, fill, evict, condense	Move from two sections to three for VL datasets; add optional heap lifecycle; wire gather/scatter to the H5T chunk-local backend.
Overwrite support	SCC VL write path and background buffer usage	Ensure old fixed descriptors are preserved for overwrite so the callback path can free old local objects.

Cache admission	SCC request sizing and batching path	Reads use exact section sizes from chunk-index metadata; writes use conservative upper bounds derived from incoming VL data and existing chunk state.
v1 fallback policies	Encode-in-place and vector I/O paths	Force fallback or report unavailable for VL structured chunks in v1.

8.6.2 H5T Chunk-Local VL Backend

The current disk VL backend is the pattern to mirror. It uses a callback table, a 4 + blob_id_size descriptor, and a storage reference in the bytes after the first four length bytes. The chunk-local backend keeps that shape and replaces only the storage-end operations.

```
/* New disk-like backend for structured chunk VL */
static const H5T_vlen_class_t H5T_vlen_chunk_g = {
    H5T__vlen_chunk_getlen, /* getlen */
    NULL, /* getptr */
    H5T__vlen_chunk_isnull, /* isnull */
    H5T__vlen_chunk_setnull, /* setnull */
    H5T__vlen_chunk_read, /* read */
    H5T__vlen_chunk_write, /* write */
    H5T__vlen_chunk_delete /* del */
};
```

The chunk-local heap must not be stored permanently in the datatype object. A datatype may be shared across many chunks and operations, while the heap is specific to one decoded structured chunk. Therefore, SCC gather/scatter should provide the current chunk heap to H5T through operation-local context while conversion is running. H5T callbacks borrow that context only for the duration of the conversion.

The exact internal plumbing for storing this temporary context should be chosen during implementation. The design requirement here is that the context has operation lifetime: it must be installed before invoking H5T conversion for one structured-chunk operation and removed on all success and failure exit paths.

The backend should use the operation-local context described earlier in the data-structure section. The expected support functions are:

```
/*
 * Install the chunk-local VL context for the current structured-chunk
 * conversion operation.
 */
```

```

herr_t H5T_push_vlen_chunk_ctx(H5F_t *f, H5HG_heap_t **heap, size_t
                                ref_nbytes);

/*

* Remove the current chunk-local VL context after conversion finishes. This
* must be called on both success and failure paths.

*/

herr_t H5T_pop_vlen_chunk_ctx(void);

/*

* Return the currently installed chunk-local VL context for use by the
* H5T_vlen_chunk_g callback table.

*/

const H5T_vlen_chunk_ctx_t *H5T__vlen_chunk_get_ctx(void);

/* Package-private helper: patch an operation-scoped file datatype copy */
herr_t H5T_patch_vlen_chunk_local(H5T_t *dt, H5VL_object_t *file);

/* Package-private helper: recursive delete of a file-side element for erase
*/

herr_t H5T__vlen_delete_file_elem(const H5T_t *dt, H5VL_object_t *file, void
*elem);

```

The reference-field helper functions should be package-private and should take the context so they can zero and validate the entire blob-ID-sized field.

```

static herr_t H5T__vlen_chunk_ref_encode(const H5T_vlen_chunk_ctx_t *ctx,
uint8_t *p, size_t idx);

static herr_t H5T__vlen_chunk_ref_decode(const H5T_vlen_chunk_ctx_t *ctx,
const uint8_t *p, size_t *idx);

static herr_t H5T__vlen_chunk_ref_setnull(const H5T_vlen_chunk_ctx_t *ctx,
uint8_t *p);

static htri_t H5T__vlen_chunk_ref_isnull(const H5T_vlen_chunk_ctx_t *ctx,
const uint8_t *p);

```

The first implementation uses a simple local-reference encoding that preserves the current disk VL descriptor size. The descriptor still contains the four-byte sequence length followed by the blob-ID-sized reference field. For the first implementation, the reference field encodes an H5HG-style local heap object index and zeros the remaining unused bytes.

This encoding does not introduce a new 32-bit heap-object index capacity. The chunk-local heap is intended to reuse the H5HG-style heap-object encoding as much as practical, and that encoding stores the per-collection object id in a two-byte field. Therefore, the valid local object-index range is the range supported by the selected H5HG-style heap encoding. The encode helper should validate that the supplied local index fits that range before writing the descriptor, and the decode helper should reject nonzero reserved bytes or out-of-range references.

A future implementation that needs a wider per-chunk object-index space can potentially define a new H5_SECTION_VL encoding version or a new interpretation of the descriptor reference field. Such a change would need to widen or replace the heap-object index field in the chunk-local heap-object encoding and update the reference encode/decode/null/validation helpers. It should not require a completely new heap design if the allocation, object table, free-space object, compaction, and buffer-growth mechanics remain reusable, but it would be more than a descriptor-only change.

8.6.3 H5T Callback Pseudocode

The callback bodies should be implemented along the following lines. This is pseudocode, not final compilable code. The important design point is that the callbacks preserve the existing disk VL callback shape while redirecting payload access to the chunk-local heap supplied by operation-local context.

getlen:

The getlen callback reads only the four-byte sequence length at the beginning of the descriptor. It does not inspect the local heap reference and does not require the heap to exist.

```
static herr_t
H5T_vlen_chunk_getlen(H5VL_object_t H5_ATTR_UNUSED *file, const void *_vl,
size_t *seq_len)
{
    const uint8_t *vl = (const uint8_t *)_vl;

    HDassert(vl);
    HDassert(seq_len);

    UINT32DECODE(vl, *seq_len);
    return SUCCEED;
}
```

isnull:

The isnull callback checks both halves of the descriptor: the sequence length and the local reference field. A descriptor is null only when the sequence length is zero and the local reference field is all zero. Mismatched encodings are treated as corrupt data.

```

static herr_t
H5T__vlen_chunk_isnull(const H5VL_object_t H5_ATTR_UNUSED *file, void *_vl,
bool *isnull)
{
    const H5T_vlen_chunk_ctx_t *ctx      = H5T__vlen_chunk_get_ctx();
    const uint8_t *vl                    = (const uint8_t *)_vl;
    size_t seq_len                        = 0;
    htri_t ref_null                       = FAIL;
    herr_t ret_value                      = SUCCEED;

    HDassert(ctx);
    HDassert(vl);
    HDassert(isnull);

    UINT32DECODE(vl, seq_len);

    if ((ref_null = H5T__vlen_chunk_ref_isnull(ctx, vl)) < 0)

        HGOTO_ERROR(H5E_DATATYPE, H5E_CANTGET, FAIL, "unable to check
        local VL reference");

    if ((seq_len == 0 && ref_null == FALSE) || (seq_len > 0 && ref_null ==
    TRUE))

        HGOTO_ERROR(H5E_DATATYPE, H5E_BADVALUE, FAIL, "corrupt chunk-
        local VL descriptor");

    *isnull = (seq_len == 0);

done:
    return ret_value;
}

```

setnull:

The setnull callback writes the null descriptor encoding. If a background descriptor is supplied, it first deletes the old file-side payload so overwrite-to-null does not leak a local heap object.

```

static herr_t
H5T__vlen_chunk_setnull(H5VL_object_t *file, void *_vl, void *_bg)
{
    const H5T_vlen_chunk_ctx_t *ctx      = H5T__vlen_chunk_get_ctx();
    uint8_t *vl                          = (uint8_t *)_vl;
    herr_t ret_value                      = SUCCEED;

    HDassert(ctx);
    HDassert(vl);

    if (_bg != NULL)
        if (H5T__vlen_chunk_delete(file, _bg) < 0)
            HGOTO_ERROR(H5E_DATATYPE, H5E_CANTREMOVE, FAIL, "unable to
            delete existing local payload");
}

```

```

        UINT32ENCODE(vl, 0);

    if (H5T__vlen_chunk_ref_setnull(ctx, vl) < 0)
        HGOTO_ERROR(H5E_DATATYPE, H5E_CANTSET, FAIL, "unable to null
        local reference");

    done:
        return ret_value;
}

```

read:

The read callback resolves a non-null local reference through the current chunk-local heap. A non-null descriptor with a missing heap is corrupt. A null descriptor is a no-op.

```

static herr_t
H5T__vlen_chunk_read(H5VL_object_t H5_ATTR_UNUSED *file, void *_vl, void
*buf, size_t len)
{
    const H5T_vlen_chunk_ctx_t *ctx      = H5T__vlen_chunk_get_ctx();
    const uint8_t *vl                  = (const uint8_t *)_vl;
    size_t seq_len                     = 0;
    size_t idx                         = 0;
    size_t buf_size                    = len;
    herr_t ret_value                   = SUCCEED;

    HDassert(ctx);
    HDassert(vl);
    HDassert(buf);

    UINT32DECODE(vl, seq_len);

    if (H5T__vlen_chunk_ref_decode(ctx, vl, &idx) < 0)
        HGOTO_ERROR(H5E_DATATYPE, H5E_CANTGET, FAIL, "unable to decode
        local reference");

    if (seq_len == 0 && idx == 0)
        HGOTO_DONE(SUCCEED);

    if (seq_len == 0 || idx == 0 || *(ctx->heap) == NULL)
        HGOTO_ERROR(H5E_DATATYPE, H5E_BADVALUE, FAIL, "corrupt chunk-
        local VL descriptor or missing heap");

    if (NULL == H5HG__read_local(ctx->f, *(ctx->heap), idx, buf,
&buf_size))
        HGOTO_ERROR(H5E_HEAP, H5E_CANTGET, FAIL, "unable to read local
        heap object");

    done:
        return ret_value;
}

```

write:

The write callback installs a new payload. It deletes any old background payload, lazily creates the chunk-local heap if needed, inserts the new payload, and writes the resulting local heap index into the

descriptor reference field. A zero-length write produces the null descriptor encoding and does not create a heap.

```
static herr_t
H5T__vlen_chunk_write(H5VL_object_t *file, const H5T_vlen_alloc_info_t
H5_ATTR_UNUSED *vl_alloc_info, void *_vl, void *buf, void *_bg, size_t
seq_len, size_t base_size)
{
    const H5T_vlen_chunk_ctx_t *ctx      = H5T__vlen_chunk_get_ctx();
    uint8_t                    *vl        = (uint8_t *)_vl;
    size_t                     idx         = 0;
    herr_t                     ret_value = SUCCEED;

    HDassert(ctx);
    HDassert(vl);

    if (_bg != NULL)
        if (H5T__vlen_chunk_delete(file, _bg) < 0)
            HGOTO_ERROR(H5E_DATATYPE, H5E_CANTREMOVE, FAIL, "unable to
            delete existing local payload");

    UINT32ENCODE(vl, seq_len);

    if (seq_len == 0) {
        if (H5T__vlen_chunk_ref_setnull(ctx, vl) < 0)
            HGOTO_ERROR(H5E_DATATYPE, H5E_CANTSET, FAIL, "unable to
            null local reference");
    }
    else {
        if (*(ctx->heap) == NULL)
            if (NULL == (*(ctx->heap) = H5HG__create_local(ctx->f,
            H5HG_MINSIZE)))
                HGOTO_ERROR(H5E_HEAP, H5E_CANTINIT, FAIL, "unable to
                create chunk-local heap");

        if (H5HG__insert_local(ctx->f, *(ctx->heap), seq_len *
        base_size, buf, &idx) < 0)
            HGOTO_ERROR(H5E_HEAP, H5E_CANTINSERT, FAIL, "unable to
            insert local heap object");

        if (H5T__vlen_chunk_ref_encode(ctx, vl, (uint32_t)idx) < 0)
            HGOTO_ERROR(H5E_DATATYPE, H5E_CANTSET, FAIL, "unable to
            encode local reference");
    }

done:
    return ret_value;
}
```

delete:

The delete callback removes the referenced local heap object but does not free the heap object itself. Heap ownership remains with SCC/H5D. After the conversion operation finishes, SCC/H5D may free the heap if it has become empty.

```
static herr_t
H5T__vlen_chunk_delete(H5VL_object_t H5_ATTR_UNUSED *file, void *_vl)
```

```

{
    const H5T_vlen_chunk_ctx_t *ctx      = H5T__vlen_chunk_get_ctx();
    uint8_t *v1                         = (uint8_t *)_v1;
    size_t seq_len                      = 0;
    size_t idx                          = 0;
    hbool_t heap_empty                  = FALSE;
    herr_t ret_value                     = SUCCEED;

    HDassert(ctx);
    HDassert(v1);

    UINT32DECODE(v1, seq_len);

    if (H5T__vlen_chunk_ref_decode(ctx, v1, &idx) < 0)
        HGOTO_ERROR(H5E_DATATYPE, H5E_CANTGET, FAIL, "unable to decode
        local reference");

    if (seq_len == 0 && idx == 0)
        HGOTO_DONE(SUCCEED);

    if (seq_len == 0 || idx == 0 || *(ctx->heap) == NULL)
        HGOTO_ERROR(H5E_DATATYPE, H5E_BADVALUE, FAIL, "corrupt chunk-
        local VL descriptor or missing heap");

    if (H5HG__remove_local(ctx->f, *(ctx->heap), idx, &heap_empty) < 0)
        HGOTO_ERROR(H5E_HEAP, H5E_CANTREMOVE, FAIL, "unable to remove
        local heap object");

    /*
     * Do not free ctx->heap here. SCC owns heap lifetime.
     * gather/erase/condense/encode may free the heap after the operation
     * if it has become empty.
     */

done:
    return ret_value;
}

```

The final implementation should use the existing HDF5 error-handling style for cleanup labels so that pushed context, temporary datatypes, and partially allocated heap state are released on failure.

H5T__conv_vlen() itself should remain structurally unchanged. The H5T work is concentrated in three additions:

1. the new callback table,
2. the temporary-datatype patch helper,
3. the operation-local context API.

The old H5T_vlen_disk_g path remains intact for the legacy descriptor-to-BLOB-to-global-heap path.

8.6.4 Backend Selection

The selected backend-selection strategy is to patch a temporary file datatype copy used only during structured-chunk gather/scatter. This is the least invasive path because it does not introduce a new

datatype location enum and does not permanently mutate the dataset's shared file datatype. The private patched datatype may be created per operation or cached as non-shared structured-chunk state. In both cases, the current chunk heap is supplied separately through operation-local context.

Backend-selection option	Description	Pros	Cons	Initial Implementation Status
New H5T_LOC_CHUNK	Add a new datatype-location mode.	Explicit semantic marker.	Wider ripple through H5T location logic.	Not selected.
Reuse H5T_LOC_DISK with new setter	Keep disk location but special-case structured chunks.	Smaller ripple than a new enum.	Still risks mutating real datatype state unless carefully scoped.	Not selected.
Temporary patched datatype	Copy file type, patch VL nodes to H5T_vlen_chunk_g, destroy after conversion.	Minimal semantic disruption; easy rollback; no permanent mutation.	Requires an internal patch helper and temporary type copy.	Selected.

8.6.5 Operation-Local Context Passing

The selected context-passing strategy is an operation-local VL chunk context. The implementation may use the least invasive mechanism that provides the required lifetime, cleanup behavior, and reentrancy guarantees.

The required behavior is:

Install context before structured-chunk H5T conversion begins, make the current decoded chunk heap available to H5T_vlen_chunk_g callbacks, allow write callbacks to lazily create the heap by updating the decoded chunk's vl_heap field, remove the context on every success and failure exit path, and avoid storing chunk-specific heap state in the shared datatype object.

This selected approach is operation-local context. It has the correct lifetime, avoids datatype contamination, and supports lazy heap creation. Static or TLS state is not selected because nested operations, recursion, and cleanup are harder to reason about. A permanent datatype field is not selected because datatypes can be shared across chunks and operations. A conversion-path argument extension is not selected for this version because it would require a larger ripple through the conversion machinery.

The recursive file-side delete helper should mirror the existing recursive reclaim logic conceptually but operate on file-side descriptors and chunk-local callbacks. It is needed for selective erase and for any other path that deletes a subset of elements without discarding the whole chunk.

8.6.6 H5HG Local Heap Reuse

The reusable H5HG pieces are:

- H5HG_heap_t
- heap buffer
- object table
- reserved free-space object at index 0
- allocation from free space
- compaction on removal
- heap extension / growth
- pointer repair after growth

The important RAM-format decision is that the chunk-local heap should look like an H5HG heap while decoded. The decoded chunk should own an H5HG_heap_t. The heap->chunk buffer should contain the contiguous local heap image in memory, and heap->obj[] should describe the objects inside that buffer. Descriptor local-reference values should resolve to entries in heap->obj[].

This mirrors the current H5HG RAM model and keeps allocation, lookup, removal, extension, compaction, and pointer repair close to existing H5HG behavior. The difference is ownership and persistence: the local heap is owned by the decoded structured chunk and is serialized into H5_SECTION_VL, rather than being protected as an independent metadata-cache object with its own file address.

The pieces that should not be reused in the local chunk-heap path are:

- metadata-cache protection
- CWFS tracking
- file-address identity
- independent metadata-object lifecycle
- file-space allocation for standalone global heap objects

The local helpers should live alongside current H5HG internals and reuse internal heap mechanics as directly as practical.

```

H5HG_heap_t *H5HG__create_local(H5F_t *f, size_t init_size);

herr_t      H5HG__free_local(H5HG_heap_t *heap);

herr_t      H5HG__insert_local(H5F_t *f, H5HG_heap_t *heap, size_t size,
                               const void *obj, size_t *idx_out);

void        *H5HG__read_local(H5F_t *f, const H5HG_heap_t *heap, size_t idx,
                               void *object, size_t *buf_size);

herr_t      H5HG__remove_local(H5F_t *f, H5HG_heap_t *heap, size_t idx,
                               hbool_t *heap_empty);

herr_t      H5HG__extend_local(H5F_t *f, H5HG_heap_t *heap, size_t need);

htri_t      H5HG__is_empty_local(const H5HG_heap_t *heap);

herr_t      H5HG__encode_local(H5F_t *f, const H5HG_heap_t *heap,
                               uint8_t **buf, size_t *nbytes, size_t *alloc_size);

herr_t      H5HG__decode_local(H5F_t *f, const uint8_t *buf, size_t nbytes,
                               H5HG_heap_t **heap_out);

```

8.6.7 H5HG Local Helper Algorithms

The local H5HG helpers should reuse the internal heap mechanics of H5HG while avoiding the file-addressed metadata-object lifecycle. These helpers operate on a chunk-owned `H5HG_heap_t *` and are used only while encoding, decoding, reading, writing, or deleting payloads inside one structured chunk.

`H5HG__create_local()`:

`H5HG__create_local()` creates an in-memory H5HG heap object without assigning it a file address, registering it with the metadata cache, or allocating standalone heap storage in the file. The resulting heap is owned by the decoded chunk object.

```

H5HG__create_local(f, init_size):
    size = MAX(init_size, H5HG_MINSIZE)

    allocate H5HG_heap_t
    zero cache-only fields
    set heap->addr = HADDR_UNDEF

    allocate heap->chunk of size bytes
    allocate heap->obj table

    initialize H5HG collection header in heap->chunk
    reserve obj[0] as free-space entry

    set heap->nused = 1
    set heap->size = size

```

```
return heap
```

H5HG__insert_local():

H5HG__insert_local() inserts one payload object into the chunk-local heap. It should reuse the existing H5HG allocation logic where possible, but it should grow the local heap buffer directly instead of allocating a new standalone global heap object in the file.

```
H5HG__insert_local(f, heap, size, obj, idx_out):
    need = H5HG_SIZEOF_OBJHDR(f) + H5HG_ALIGN(size)

    if heap->obj[0].size < need:
        H5HG__extend_local(f, heap, need - heap->obj[0].size)

    idx = H5HG__alloc(f, heap, size, &heap_flags)

    memcpy(heap->obj[idx].begin + H5HG_SIZEOF_OBJHDR(f), obj, size)

    *idx_out = idx

    return SUCCEED
```

H5HG__read_local():

H5HG__read_local() resolves a local heap object index to its payload bytes. If the caller does not provide a destination buffer, the helper allocates one. If the caller provides a buffer, the helper verifies that it is large enough before copying.

```
H5HG__read_local(f, heap, idx, object, buf_size):
    validate idx != 0
    validate idx < heap->nused
    validate heap->obj[idx].begin != NULL

    payload = heap->obj[idx].begin + H5HG_SIZEOF_OBJHDR(f)
    size     = heap->obj[idx].size

    if object == NULL:
        allocate size bytes
        copy payload
        *buf_size = size
        return new buffer
    else:
        verify *buf_size >= size
        copy payload into object
        *buf_size = size
        return object
```

H5HG__remove_local():

H5HG__remove_local() removes one payload object from the chunk-local heap. The helper should update the object table and compact the heap buffer so the eventual H5_SECTION_VL image does not preserve unnecessary dead space.

```
H5HG__remove_local(f, heap, idx, heap_empty):
    validate idx

    compute removed span including object header and alignment

    shift later object begin pointers left by removed span
    memmove trailing bytes left inside heap->chunk

    expand free-space object obj[0]
    rewrite free-space object header if space permits

    zero heap->obj[idx]

    *heap_empty = H5HG__is_empty_local(heap)

    return SUCCEED
```

H5HG__extend_local():

H5HG__extend_local() grows the in-memory heap buffer when the free-space object cannot satisfy an insertion. After reallocation, it must repair object-table pointers because the heap buffer base address may have changed.

```
H5HG__extend_local(f, heap, need):
    choose new_size >= heap->size + need

    old_chunk = heap->chunk
    realloc heap->chunk to new_size
    zero new tail bytes

    repair all obj[u].begin pointers relative to new base

    update heap->size
    expand obj[0] free-space entry
    rewrite heap/free-space headers

    return SUCCEED
```

H5HG__is_empty_local():

H5HG__is_empty_local() reports whether the local heap contains any allocated payload objects. The reserved free-space object at index 0 does not count as a payload object.

```
H5HG__is_empty_local(heap):
    return TRUE when no allocated payload objects remain
```

H5HG__encode_local():

H5HG__encode_local() serializes a non-empty chunk-local heap into the logical H5_SECTION_VL payload. Empty heaps encode as zero bytes. The first implementation should try to reuse the existing H5HG serialized heap image directly and add a minimal wrapper only if the reused image is not self-describing enough for H5_SECTION_VL.

```
H5HG__encode_local(f, heap, buf, nbytes, alloc_size):
    if heap == NULL or H5HG__is_empty_local(heap):
        *buf = NULL
        *nbytes = 0
        *alloc_size = 0
        return SUCCEED

    compact or prepare the heap image according to the vl compaction
    policy

    if the existing H5HG serialized image is sufficient as a standalone
    H5_SECTION_VL payload:
        encode the H5HG-style heap image directly into *buf
    else:
        encode a minimal local wrapper containing a version/format
        marker
        followed by the H5HG-style heap image

    *nbytes = encoded section size
    *alloc_size = allocated buffer size

    return SUCCEED
```

H5HG__decode_local():

H5HG__decode_local() reconstructs a chunk-local heap from the logical H5_SECTION_VL payload. A zero-sized VL section produces heap_out = NULL. A nonzero VL section must be validated before the reconstructed heap is attached to the decoded chunk object.

```
H5HG__decode_local(f, buf, nbytes, heap_out):
    if nbytes == 0:
        *heap_out = NULL
        return SUCCEED

    determine whether buf begins with a direct H5HG-style heap image or
    a minimal local wrapper

    validate the heap image/version information

    reconstruct H5HG_heap_t from the heap image
    rebuild heap->obj table if the reused deserialize path does not do
    so

    *heap_out = heap

    return SUCCEED
```


The recommended reuse boundary is the internal heap mechanics, not the file-addressed metadata-object lifecycle. The local path must not call file-space allocation, metadata-cache protect/unprotect, CWFS maintenance, or any code that assumes `heap->addr` names a standalone metadata object.

Before deeper refactoring, the implementation should test whether the existing H5HG collection image can be used directly as the `H5_SECTION_VL` payload. If direct image reuse works cleanly, it is preferred because the existing H5HG image already carries heap-format information. If it does not, `H5_SECTION_VL` may add a minimal wrapper before the H5HG body. In either case, the wrapper decision is local to the section format and should not affect H5T descriptor semantics.

8.6.8 SCC and H5D Structured-Chunk Changes

The structured-chunk code currently assumes two sections in several encode/decode paths: a selection section and a fixed-data section. For VL datasets, that logic must expand to the three-section model defined earlier:

```
H5_SECTION_SELECTION
H5_SECTION_FIXED
H5_SECTION_VL
```

The main SCC/H5D work is to make the decoded chunk object carry optional VL heap state, make encode/decode understand the optional VL section, and route gather/scatter through the chunk-local H5T backend when the dataset file datatype contains VL.

Section Setup

The structured-chunk storage message already tracks the number of sections and the subset of sections that contain metadata. For fixed-size structured chunks, the existing behavior is two sections, with only the selection section treated as metadata-bearing. For VL structured chunks, the implementation should use three sections, and all three should be metadata-bearing.

Conceptually, `H5D__struct_chunk_set_sizes()` should become:

```
has_vlen_type = H5T_detect_class(dset->shared->type, H5T_VLEN, false);

if (has_vlen_type) {
    dset->shared->layout.storage.u.struct_chunk.nsects =
        H5_SECTION_NUM;
    dset->shared->layout.storage.u.struct_chunk.nsects_md =
        H5_SECTION_NUM;

    dset->shared->layout.storage.u.struct_chunk.seq_sects_md[0] =
        H5_SECTION_SELECTION;
    dset->shared->layout.storage.u.struct_chunk.seq_sects_md[1] =
        H5_SECTION_FIXED;
    dset->shared->layout.storage.u.struct_chunk.seq_sects_md[2] =
        H5_SECTION_VL;
}
else {
```

```

dset->shared->layout.storage.u.struct_chunk.nsects      = 2;
dset->shared->layout.storage.u.struct_chunk.nsects_md = 1;

dset->shared->layout.storage.u.struct_chunk.seq_sects_md[0] =
H5_SECTION_SELECTION;
}

```

This relies on H5_SECTION_NUM representing the number of real structured-chunk sections. H5_SECTION_UNKNOWN is not a real section and must not be counted.

For VL datasets, H5_SECTION_FIXED is metadata-bearing because it stores root VL descriptors, and H5_SECTION_VL is metadata-bearing because it stores the serialized heap structure and may contain nested VL descriptors inside heap payloads.

Section Indexing

All hardcoded two-section assumptions must be replaced with named section indices. Current paths that use values such as:

```

offset[1]
unfilt_size[0]
unfilt_size[1]
filt_mask[0]
filt_mask[1]

```

should be updated to use:

```

offset[H5_SECTION_SELECTION]
offset[H5_SECTION_FIXED]
offset[H5_SECTION_VL]

unfilt_size[H5_SECTION_SELECTION]
unfilt_size[H5_SECTION_FIXED]
unfilt_size[H5_SECTION_VL]

filt_mask[H5_SECTION_SELECTION]
filt_mask[H5_SECTION_FIXED]
filt_mask[H5_SECTION_VL]

```

Filter switch statements that currently assert or fail on H5_SECTION_VL must add a real VL-section case.

Decoded Chunk State

H5D_chunk_cache_mem_t or the equivalent decoded chunk representation must add and consistently maintain:

```

H5HG_heap_t *vl_heap;
size_t      vl_nbytes;
size_t      vl_alloc_size;

```

H5D__struct_chunk_new_chunk() should initialize the VL state to empty:

```
chk->vl_heap      = NULL;
chk->vl_nbytes     = 0;
chk->vl_alloc_size = 0;
```

It should not allocate an empty heap. The heap should be created lazily only when a non-null VL payload is written.

All allocation and cleanup paths must initialize and release the new VL fields. This includes new chunk creation, decode, condense, encode failure paths, gather/scatter failure paths, fill, erase, evict, and any partial decode path that can allocate a decoded chunk object.

Decode Path

H5D__struct_chunk_decode() must split the encoded chunk image into three logical sections for VL datasets. It should compute section boundaries from offset[H5_SECTION_*] and the total encoded chunk size.

For VL datasets, decode should:

```
decode H5_SECTION_SELECTION into the chunk selection state
copy H5_SECTION_FIXED into the decoded fixed-data buffer
decode nonzero H5_SECTION_VL bytes into chk->vl_heap
leave chk->vl_heap = NULL when H5_SECTION_VL is zero-sized
```

Filters and checksums should be applied or verified section by section using the existing SCC machinery. After checksum metadata is removed, only logical section bytes should be passed to the selection decoder, fixed-data consumer, or local heap decoder.

Encode Path

H5D__struct_chunk_encode() performs the inverse operation. For VL datasets, encode should:

```
produce logical selection bytes
produce logical fixed descriptor bytes
produce logical VL bytes if chk->vl_heap is non-NULL and non-empty
produce zero VL bytes if chk->vl_heap is NULL or empty
protect metadata-bearing sections using SCC checksum machinery
apply section filters if configured
concatenate sections in H5_SECTION_* order
record offset[], unfilt_size[], and filt_mask[] by H5_SECTION_* index
```

If the chunk has no defined values, encode should not persist a chunk entry only to preserve an empty VL heap. A zero-sized H5_SECTION_VL is valid inside an existing chunk, but an entirely empty chunk should be omitted.

Gather and Scatter

H5D__struct_chunk_gather_mem() and H5D__struct_chunk_scatter_mem() are the SCC/H5D points where H5T integration happens.

For VL datasets, both paths should:

```
create a temporary copy of the file datatype
patch file-side VL nodes to use the chunk-local VL backend
install the operation-local VL chunk context
call the existing conversion path
pop the chunk-local context on all exit paths
free the temporary datatype copy on all exit paths
```

The write side must also preserve background file-side descriptors when overwriting existing VL values. Those background descriptors allow the H5T chunk-local callbacks to delete old local heap objects before installing new payloads.

Erase and Fill

H5D__struct_chunk_erase_values() needs special handling for VL datasets.

For a full-chunk erase, SCC/H5D can drop the chunk and free the entire chunk-local heap wholesale. No per-object recursive cleanup is needed because the heap is local to the chunk.

For a partial erase, SCC/H5D must invoke the file-side recursive VL delete helper on each erased fixed record before removing that record from the fixed section and updating the selection section. This is required for nested VL, where child descriptors may be stored inside heap payloads.

H5D__struct_chunk_fill() should write null VL fill values as null descriptors without creating a heap. Non-null VL fill values should be routed through the same temporary patched-type gather path used for ordinary writes.

Cleanup, Condense, and Fallback Paths

H5D__struct_chunk_evict() should free vl_heap if it is non-NULL.

H5D__struct_chunk_condense() and related size-finalization paths should include optional VL bytes in size calculations. If the heap is empty, they should normalize the decoded state back to:

```
chk->vl_heap      = NULL;
chk->vl_nbytes     = 0;
chk->vl_alloc_size = 0;
```

H5D__struct_chunk_encode_in_place() should force fallback for VL datasets. The optional VL section and heap compaction make in-place encoding unsafe for the initial implementation.

H5D__struct_chunk_vector_read() and H5D__struct_chunk_vector_write() should report that the optimized vector path is unavailable for VL structured chunks in the initial implementation.

Error paths should follow the same ownership rules as normal cleanup. Temporary datatype copies, installed operation-local context, temporary section buffers, and partially allocated VL heap state must be released before returning failure.

8.6.9 File-Format Persistence Points

The implementation must update the existing sectioned-storage persistence paths rather than inventing a new dataset layout message. In particular, the object-header data layout message version 5 encode/decode path must preserve:

- sectioned data type

- sectioned storage type

- number of sections

- number of sections containing metadata

- list of metadata-bearing section numbers

For sparse VL structured chunks, the sectioned data type must indicate variable-length data. The number of sections must include H5_SECTION_VL, and the metadata-bearing section list must include H5_SECTION_SELECTION, H5_SECTION_FIXED, and H5_SECTION_VL. Dense VL should likewise include H5_SECTION_FIXED and H5_SECTION_VL, but does not require a selection section unless sparse behavior is also selected.

The chunk-index metadata path must also preserve per-section location and size information for H5_SECTION_VL, including section offsets, unfiltered sizes, and filter masks where applicable. This is required so the chunk image can be split into sections, defiltered, checksum-verified, and decoded correctly after the file is reopened.

8.6.10 Cache Admission and Request Sizing

The cache-admission or request-sizing path is another required SCC code area.

Reads can compute exact chunk sizes from chunk-index metadata because the total encoded chunk size and section offsets are available.

Writes should use a conservative upper bound based on:

- existing decoded chunk state
- incoming fixed descriptor bytes
- incoming VL payload sizes

- potential new heap objects
- possible compaction result

The estimate may be pessimistic before compaction and heap reuse, but it should still be materially better than using an unrelated dense-chunk upper bound. Ordinary eviction and batching logic can then decide whether to process the full request or split it.

8.7 Integrity, Filtering, and Section Metadata Policy

The integrity rule for this implementation is that every metadata-bearing section present in the encoded structured-chunk image must be protected by the structured-chunk section checksum mechanism. The VL implementation should not introduce a separate H5HG-local or VL-heap-internal checksum mechanism.

For VL structured chunks, the metadata-bearing sections are:

H5_SECTION_SELECTION:

Metadata describing which logical elements are defined.

H5_SECTION_FIXED:

Metadata for VL datasets because it stores root descriptors that reference chunk-local heap objects.

H5_SECTION_VL:

Metadata-bearing when non-zero because it contains the serialized heap structure and may contain nested VL descriptors embedded in heap objects.

The structured-chunk storage message already tracks metadata-bearing sections with `nsects_md` and `seq_sects_md[]`. For VL datasets, the implementation should set:

```
nsects_md = 3
seq_sects_md[0] = H5_SECTION_SELECTION
seq_sects_md[1] = H5_SECTION_FIXED
seq_sects_md[2] = H5_SECTION_VL
```

For fixed-size datasets, the existing behavior remains:

```
nsects_md = 1
seq_sects_md[0] = H5_SECTION_SELECTION
```

Filtering should remain a structured-chunk section operation. The VL implementation should produce logical section payloads for the selection metadata, fixed records/root descriptors, and optional VL heap bytes. SCC should then apply filters according to the same section-level machinery used for other structured chunks.

The decode verification order should be:

1. Use `offset[]` and the encoded chunk size to isolate section byte ranges.

2. Reverse filters on each section as required by SCC.
3. Verify the SCC checksum for each metadata-bearing section.
4. Remove checksum bytes or checksum metadata from the logical section payload according to SCC rules.
5. Pass logical section bytes to the selection decoder, fixed-section consumer, or `H5HG__decode_local()`.

The existing implementation stores and verifies a checksum for the encoded selection metadata. The VL implementation should extend the same metadata-section checksum convention to `H5_SECTION_FIXED` and `H5_SECTION_VL` for VL datasets. The exact helper code may still be factored differently during implementation, but the design requirement is fixed: metadata-bearing VL structured-chunk sections are protected by SCC, not by a separate local-heap checksum.

8.8 Initial Implementation Policy Decisions

This section collects the policy decisions for the first implementation. These choices are intended to minimize risk and implementation scope while leaving an upgrade path for later formats.

8.8.1 Initial Decision Table

Decision point	Alternatives considered	Selected choice	Reason
Descriptor size	Keep 4 + blob_id_size; shrink to 4 + 4; introduce new tagged descriptor.	Keep 4 + blob_id_size.	Smallest ripple through H5T, buffers, and conversion paths.
File-side backend selection	New location enum; permanent location mutation; temporary patched datatype.	Temporary patched datatype.	Localized change; no mutation of shared datatype state.
Context passing	Operation-local context; TLS/static; permanent datatype field; conversion path extension.	Operation-local context.	Correct per-operation lifetime; no datatype contamination.
Empty heap encoding	Always emit minimal header; zero-size VL section; keep empty heap blob even if empty.	Zero-size <code>H5_SECTION_VL</code> and delete empty heap.	Matches optional-heap policy and avoids materializing useless heap state.

Section integrity	Mixed SCC and heap-internal checksums; heap-internal only; SCC protection for all metadata sections.	SCC protection for all metadata sections.	One mechanism, one decode order, no duplicate checksum logic unless required.
Advanced write path	Enable encode-in-place/vector I/O immediately; force fallback first.	Force fallback for v1.	Keeps first implementation focused on one correct path.
Heap image versioning	Force new wrapper; direct H5HG image only; direct image if sufficient, wrapper if needed.	Direct H5HG image if sufficient; otherwise minimal wrapper.	Maximizes H5HG reuse without blocking future format changes.

8.8.2 Empty Chunk and Empty Heap Policy

The empty-chunk and empty-heap policy is normative for the first implementation.

An empty heap is not a reason to keep a heap section alive. A chunk with fixed data but no non-null VL payloads keeps selection and fixed sections but encodes H5_SECTION_VL as zero bytes. A chunk with no defined values at all should not exist in the chunk index.

This policy must be carried consistently through:

- Gather
- Erase
- Fill
- Condense
- Encode
- Evict

The practical rule is:

- defined values + non-null VL payloads:
 - store selection, fixed, and non-empty VL sections
- defined values + no non-null VL payloads:

store selection and fixed sections

encode H5_SECTION_VL as zero bytes

- no defined values:

omit the chunk entry entirely

8.8.3 Versioning Policy

The versioning policy is local to H5_SECTION_VL, not a new global datatype tag. Legacy global-heap disk VL descriptors remain unchanged outside the structured-chunk VL path.

For the first implementation, the code should first determine whether the reused H5HG heap image already provides sufficient version information. If so, H5_SECTION_VL may store the H5HG-style heap image directly. If not, H5_SECTION_VL should add a minimal wrapper containing a format/version marker before the heap body.

Future revisions may change the local heap body encoding, descriptor-reference width, or compaction strategy. The decode path must therefore be version-aware, even if the first format supports only one concrete encoding.

8.8.4 Compatibility Policy

The existing global-heap VL path remains available and unchanged for legacy files and for layouts that do not use the structured-chunk VL path. The new chunk-local backend is selected only while structured-chunk gather/scatter is processing a VL dataset through temporary patched datatype and operation-local context.

legacy disk VL:

H5T_vlen_disk_g

-> VOL blob callback

-> native/global heap path

structured-chunk VL:

temporary patched file datatype

-> H5T_vlen_chunk_g

-> current chunk-local H5HG heap

-> H5_SECTION_VL

Contiguous VL datasets and existing global-heap-backed datasets are not converted by the first implementation. They continue to use the existing descriptor-to-blob/global-heap mechanism.

8.9 Implementation Order

The implementation should be staged so that each step has a testable result.

1. H5HG local heap helpers

Add create/free/insert/read/remove/extend/is_empty helpers that operate on an H5HG_heap_t * without metadata-cache or file-address ownership.

2. H5HG local encode/decode

Add encode/decode helpers for H5_SECTION_VL. First test whether a direct H5HG-style heap image is sufficient. Add a minimal wrapper only if direct reuse is not sufficient.

3. Decoded chunk object

Add optional vl_heap state to the structured-chunk decoded representation. Initialize it to NULL for new chunks and free it during evict/cleanup.

4. SCC three-section lifecycle

Extend set-sizes, decode, encode, condense, and evict so VL datasets reserve H5_SECTION_SELECTION, H5_SECTION_FIXED, and H5_SECTION_VL. H5_SECTION_VL may encode as zero bytes.

5. H5T chunk-local backend

Add H5T_vlen_chunk_g and local-reference encode/decode helpers. Keep H5T__conv_vlen() structurally unchanged.

6. Operation-local context

Add temporary operation-local context so H5T chunk-local callbacks can access the current decoded chunk heap during scatter/gather. The context must be installed before conversion and removed on all success and failure paths.

7. Gather/scatter integration

Use a temporary patched file datatype view/copy to route file-side VL nodes to H5T_vlen_chunk_g, push operation-local context, call the existing conversion path and pop context on all success and failure paths. The shared dataset datatype must remain unchanged.

8. Overwrite, erase, and fill

Preserve background descriptors for overwrite. Add or verify recursive file-side delete support for selective erase. Route non-null fill through the same chunk-local H5T path as ordinary writes.

9. Disable advanced paths for first implementation

Force fallback for encode-in-place and vector I/O on VL structured chunks until the ordinary path is correct.

10. Tests

Start with one-chunk VL string or VL integer write/read/close/reopen. Then add overwrite, null values, erase, fill, compound-with-VL, nested VL, filters/checksums, corrupt descriptor cases, missing/corrupt heap section cases, compatibility with the legacy VL path, and multi-chunk requests.

9 Acknowledgement

This work is supported by the U.S. Department of Energy, Office of Science under award number DE-SC0023583 for Phase II SBIR project “Supporting Sparse Data in HDF5”

10 References

1. The HDF Group. “HDF5 Documentation,” <http://www.hdfgroup.org/HDF5/doc/doc-info.html> (June 14, 2010).

2. The HDF Group. “HDF5 Documentation,” https://support.hdfgroup.org/documentation/hdf5/latest/1b_datatypes.html (June 14, 2010).

3. The HDF Group. “The HDF5 Field Guide,” https://portal.hdfgroup.org/documentation/hdf5/latest/group_v1_e_n.html (June 14, 2010).

4. Clay Carper. “RFC: Shared Chunk Cache Design,” [SparseHDF5/design_docs/05_19_2026_Shared_Chunk_Cache_RFC.pdf at main · LifeboatLLC/SparseHDF5 · GitHub](#) (May 19, 2026).

5. The HDF Group. “HDF5 File Format Specification Version 3.1,” https://github.com/LifeboatLLC/SparseHDF5/blob/main/design_docs/HDF5%20File%20Format%20Specification%20Version%203.1%202025-01-13-RFCv22.html (January 13, 2025).

11 Revision History

<i>April 30, 2026</i>	Version 0 circulated for comment within Lifeboat, LLC.
<i>June 17, 2026</i>	Version 0.10 circulated for review and comment within Lifeboat, LLC.